Tutorial II

Desarrollo y Publicación de una aplicación web con Backend y Frontend en Microsoft Azure mediante Docker, alojada en GitHub con despliegue automático con GitHub Actions

Universidad Adolfo Ibáñez

Taller de Innovación y Emprendimiento Tecnológico

Profesor: Martín Céspedes Escuti

Ayudante Instructor: Artemio Araya

17 de octubre de 2026

Proyecto de referencia:

artemioadaysolvers/Tutorial-Web-II (artemioadaysolvers, s. f.).

Leguajes principales:

- TypeScript para Frontend

- Python para Backend

Principales Herramientas sobre los Lenguajes

- next.js

- node.js

- three.js

Objetivo: Explicar, paso a paso, el proceso de configuración, integración y despliegue de una aplicación web que consiste en un typewritter de los años 80 con sistema de Login utilizando los servicios de Microsoft Azure y GitHub. El procedimiento contempla la creación de una imagen Docker para instalar sus dependencias, su almacenamiento en un registro privado de Azure Container Registry, la implementación y ejecución del contenedor en Azure App Service, la configuración de Azure Cosmos DB para el almacenamiento y la manipulación de datos, y la automatización del proceso de compilación, publicación y actualización de la aplicación mediante GitHub Actions (artemioadaysolvers, s. f.; Microsoft, s. f.-i).

De esta manera, se busca establecer una infraestructura en la nube que permita alojar una aplicación web funcional, conectada a una base de datos y provista de un sistema de autenticación de usuarios, cuyo despliegue se actualice automáticamente ante modificaciones del código fuente en GitHub (artemioadaysolvers, s. f.; GitHub, s. f.-l).

Índice

Definiciones 6

Estructura de una Página Web 6

Protocolos de Comunicación 7

Backend & Frontend 10

¿Por qué hay tantos lenguajes de Programación? 12

Lenguajes de programación utilizados para desarrollar páginas web 19

GitHub & GitHub Desktop para Desarrollo Web 25

GitHub 25

GitHub Pull Request 29

GitHub Actions 29

GitHub Desktop 31

¿Cómo se relaciona GitHub con Microsoft Azure para desarrollo Web? 32

1. Integración de GitHub con Microsoft Azure para administrar y desplegar el código de una aplicación web . 32

2. Integración de GitHub con Microsoft Azure para autenticar a los usuarios de una página web 34

Microsoft Azure 35

Windows Server 37

Servicios de Microsoft Azure útiles para desarrollo Web . . . 41

App Service 41

Azure Cosmos DB en Micrsofot Azure 42

Azure Container Registry 43

Tecnología que podemos desarrollar con Microsoft Azure . . . 46

Visualización Web vs. Renderizado Nativo en el Sistema Operativo 48

Descripción general de la aplicación web 51

Publicación de la aplicación Web 52

Funcionamiento de la web 54

Estructura del Repositorio de GitHub 58

Principales Herramientas de JavaScript utilizadas 59

Tutorial 60

Crear la cuenta de Estudiante 62

Nombres que deben anotarse 63

Descargar la Aplicación Web y subir el Proyecto a GitHub 64

Crear el repositorio 66

Subir los archivos con GitHub Desktop 67

Crear el grupo de recursos y Azure Container Registry 68

Crear el grupo de recursos 68

Crear el registro de imágenes 69

Crear la cuenta de Azure Cosmos DB 70

Crear la base de datos y el contenedor en la cuenta de Azure Cosmos DB cosmosrecados2026abc 71

Obtener el URI y la clave de acceso de Azure Cosmos DB 71

Crear App Service con Docker 72

Crear la Web App 72

Obtener la URL pública de la aplicación y configurar el acceso HTTPS 73

Permitir la conexión de App Service con Cosmos 74

Configurar el login con cuentas GitHub 75

Configurar las variables de ejecución 77

Configurar la publicación automática desde GitHub 79

Habilitar las credenciales de administrador de Azure Container Registry 79

Conectar GitHub desde Azure 80

Referencias 82

Definiciones

Estructura de una Página Web

Una página web es un documento de hipertexto que forma parte de la World Wide Web, un sistema distribuido de información cuyos recursos se identifican mediante direcciones URL (*Uniform Resource Locator*). Su contenido puede integrar texto, imágenes y referencias a otros recursos. El acceso se realiza mediante una arquitectura cliente-servidor: el navegador actúa como cliente, solicita los recursos a un servidor web e interpreta los documentos recibidos para presentar su contenido al usuario (Tanenbaum & Wetherall, 2012; Fielding et al., 2022).

El hipertexto es una estructura de organización de la información que vincula documentos o secciones mediante enlaces, permitiendo acceder a contenidos relacionados
a través de una navegación no secuencial (Tanenbaum & Wetherall, 2012).

Para representar la estructura de una página web se utiliza HTML (*HyperText Markup Language*), un lenguaje de marcado que describe los elementos del documento mediante etiquetas y atributos. Las etiquetas identifican componentes como encabezados, párrafos, imágenes y enlaces, mientras que los atributos especifican propiedades adicionales. El navegador interpreta este marcado para construir la representación del documento que se presenta al usuario (WHATWG, 2026b).

La transferencia de estos recursos requiere protocolos de comunicación, entendidos como conjuntos de reglas y convenciones que regulan el intercambio de información entre entidades de una red. En la Web, HTTP (*Hypertext Transfer Protocol*) establece un modelo de solicitud y respuesta: el cliente envía una solicitud que identifica el recurso al que desea acceder y el servidor devuelve una respuesta que indica el estado de la operación e incluye, cuando corresponde, el contenido requerido. Así, el hipertexto organiza las relaciones entre documentos, HTML describe su estructura y HTTP permite su transferencia entre clientes y servidores (Tanenbaum & Wetherall, 2012; Fielding et al., 2022).

Protocolos de Comunicación

Los protocolos de comunicación establecen las reglas que permiten intercambiar e interpretar información entre sistemas conectados a una red. En la Web, HTTP (*Hypertext Transfer Protocol*) regula la comunicación entre clientes y servidores mediante un modelo de solicitud y respuesta. El cliente solicita un recurso o una operación, y el servidor devuelve una respuesta que indica el resultado e incluye, cuando corresponde, el contenido solicitado (Fielding et al., 2022).

HTTP no proporciona por sí mismo cifrado ni protección criptográfica de la información transmitida. Por ello, cuando se utiliza sin una capa de seguridad, un tercero con acceso al tráfico de la red podría leer o modificar los datos intercambiados. Esta limitación requiere mecanismos adicionales para proteger la comunicación (Rescorla, 2000).

HTTPS corresponde al uso de HTTP mediante una comunicación protegida con TLS (*Transport Layer Security*). Mantiene el modelo de solicitud y respuesta e incorpora mecanismos criptográficos para proteger los datos durante su transmisión y autenticar al servidor. TLS proporciona tres propiedades fundamentales (Fielding et al., 2022; Rescorla, 2018):

- **Confidencialidad**: protege el contenido mediante cifrado, de modo que su interceptación no permita conocer directamente la información transmitida.

- **Integridad**: permite detectar modificaciones no autorizadas de los datos durante la comunicación.

- **Autenticación**: permite comprobar la identidad del servidor con el que se establece la conexión. También puede utilizarse para autenticar al cliente cuando la configuración lo requiere.

En el uso habitual de HTTPS, la autenticación del servidor se apoya en certificados digitales. Estos vinculan una identidad, como un nombre de dominio, con una clave pública. El navegador comprueba que el certificado corresponda al dominio solicitado, se encuentre vigente y disponga de una cadena de certificación válida hasta una autoridad de confianza. Durante el establecimiento de la conexión, el servidor demuestra que posee la clave privada correspondiente (Fielding et al., 2022; Rescorla, 2018).

¿Quién emite los certificados digitales y cómo se determina su confiabilidad?

Los certificados digitales utilizados en HTTPS son emitidos y firmados por entidades denominadas autoridades de certificación. Antes de emitir un certificado, estas entidades verifican que el solicitante controle el dominio correspondiente y, según el tipo de certificado, pueden realizar comprobaciones adicionales sobre la organización solicitante. El certificado vincula el nombre del dominio con una clave pública y establece un periodo de validez. Su emisión puede realizarse mediante procedimientos automatizados (Internet Security Research Group, s. f.).

La confianza en el certificado se determina mediante su validación por parte del navegador. Cuando el usuario accede a un sitio web, el servidor presenta su certificado y los certificados intermedios necesarios para verificar su procedencia. El navegador comprueba que el certificado corresponda al dominio solicitado, se encuentre vigente y esté vinculado, mediante una cadena de firmas digitales válidas, con una autoridad certificadora raíz de confianza. Para ello, utiliza un almacén que contiene certificados raíz de autoridades reconocidas, no los certificados de todos los sitios web. Este almacén suele incluirse y actualizarse con el navegador o con el sistema operativo del dispositivo del usuario, como Windows, y también puede ser configurado por una organización. Durante el establecimiento de la conexión, el servidor demuestra que posee la clave privada asociada al certificado presentado (Fielding et al., 2022; Rescorla, 2018).

Por tanto, la confiabilidad del certificado depende de autoridades previamente reconocidas y de comprobaciones criptográficas efectuadas por el navegador. Su aceptación permite autenticar al servidor para el dominio solicitado y establecer una comunicación protegida, pero no acredita la veracidad del contenido, la honestidad del propietario ni la ausencia de vulnerabilidades en la aplicación. En consecuencia, un certificado válido no constituye una garantía de seguridad integral del sitio web (Rescorla, 2000).

¿Qué garantiza un certificado digital de confianza y cuáles son sus limitaciones?

En una conexión HTTPS correctamente establecida, estos mecanismos proporcionan las siguientes propiedades: (Rescorla, 2018)

- **Autenticación del servidor**: permite comprobar que el servidor está autorizado para identificarse mediante el dominio solicitado.

- **Confidencialidad de la comunicación**: protege mediante cifrado el contenido transmitido frente a su lectura por terceros durante el intercambio.

- **Integridad de la información transmitida**: permite detectar modificaciones no autorizadas de los datos durante la comunicación.

Sin embargo, la aceptación del certificado no garantiza que el propietario del sitio sea honesto, que sus contenidos sean verdaderos ni que los productos o servicios ofrecidos sean legítimos. Tampoco acredita que la aplicación carezca de vulnerabilidades, que los dispositivos participantes estén libres de software malicioso o que los datos permanezcan protegidos después de su recepción y almacenamiento. Una página fraudulenta puede disponer de un certificado válido para su propio dominio; por ello, la confianza en el certificado debe distinguirse de la confianza en el sitio y en sus responsables (Rescorla, 2000, 2018; OWASP Foundation, s. f.).

Backend & Frontend

Las aplicaciones web suelen organizarse en dos partes: el frontend y el backend (Microsoft, 2025a).

Esta distinción existe desde los orígenes de la Web, a comienzos de la década de 1990, aunque las responsabilidades de cada parte han evolucionado. (CERN, s. f.).

El **frontend** comprende los componentes de una aplicación web destinados a presentar la información y permitir la interacción del usuario con el sistema. Incluye los recursos que el navegador interpreta para construir la interfaz, como documentos HTML, hojas de estilo CSS, código JavaScript e imágenes. Mediante estos componentes, el usuario puede consultar contenidos, introducir datos y solicitar operaciones a los servicios de la aplicación. Desde una perspectiva arquitectónica, corresponde a las responsabilidades de presentación, que pueden separarse del procesamiento de la información y de su almacenamiento (Microsoft, 2024, 2025a).

El **backend** comprende los componentes de una aplicación web que proporcionan los servicios de procesamiento y acceso a la información desde el lado del servidor. Se encarga de atender las solicitudes de las aplicaciones cliente, ejecutar las operaciones definidas por la lógica de negocio y gestionar la comunicación con los sistemas de almacenamiento. Estas responsabilidades pueden distribuirse entre distintas capas y servicios, con el propósito de separar las funciones del sistema y facilitar su mantenimiento, seguridad y escalabilidad (Microsoft, 2025a; Fowler, 2002).

El usuario interactúa con la aplicación web mediante el *frontend*, que presenta la información y permite introducir datos y solicitar operaciones. Cuando estas requieren procesamiento en el *backend*, el cliente envía solicitudes mediante HTTP o HTTPS y recibe las respuestas correspondientes. Esta comunicación permite utilizar los servicios de la aplicación sin conocer su implementación interna (Microsoft, 2025c).

Desde la perspectiva de la seguridad, una aplicación web debe mantener sus mecanismos de protección incluso si un tercero conoce la totalidad del código fuente de su backend. Conocer cómo está implementado el sistema no debe otorgar autorización para acceder a información protegida ni ejecutar operaciones restringidas. Para ello, la seguridad debe sustentarse en mecanismos de autenticación, autorización, validación de solicitudes y protección de credenciales, y no exclusivamente en mantener oculto el código (OWASP Foundation, s. f.; Microsoft, 2023a, 2025c).

¿Por qué hay tantos lenguajes de Programación?

Los computadores utilizan procesadores construidos mediante circuitos electrónicos formados por transistores y puertas lógicas que realizan operaciones sobre señales binarias. Estos procesadores ejecutan instrucciones en lenguaje máquina, que constituye el nivel más bajo de representación de un programa ejecutable directamente por el procesador. El lenguaje máquina consiste en instrucciones codificadas mediante secuencias de bits (ceros y unos), cuyo significado está determinado por la arquitectura del conjunto de instrucciones (*Instruction Set Architecture*, ISA). Por ejemplo, las arquitecturas x86-64, ARM64 y RISC-V utilizan conjuntos de instrucciones diferentes (Nisan & Schocken, 2005).

Por lo tanto, un procesador no distingue directamente entre lenguajes de programación como JavaScript, Python o C++, ya que estos requieren mecanismos de compilación o interpretación para que sus programas puedan ejecutarse mediante instrucciones en lenguaje máquina. Estas instrucciones son decodificadas por el procesador, que utiliza sus circuitos electrónicos para realizar físicamente las operaciones correspondientes. En consecuencia, el lenguaje máquina constituye la interfaz entre el software y el procesador, que es el componente del hardware encargado de ejecutar las instrucciones de los programas mediante sus circuitos electrónicos, permitiendo que los programas desarrollados en diferentes lenguajes de programación puedan ser ejecutados por los componentes físicos del computador (Nisan & Schocken, 2005).

La primera razón por la cuál existen variedad de lenguajes de programación es la evolución histórica de la informática y la necesidad de conservar sistemas desarrollados con tecnologías anteriores. Durante las primeras décadas de la computación, los programas se escribían directamente en lenguaje máquina o mediante lenguajes ensambladores, lo que exigía un conocimiento detallado de la arquitectura del procesador y un considerable esfuerzo de programación. Para reducir esta dificultad surgieron los primeros lenguajes de alto nivel, como FORTRAN, desarrollado durante la década de 1950, que permitió expresar operaciones matemáticas mediante instrucciones más comprensibles y traducirlas automáticamente a lenguaje máquina. Estos lenguajes representaron importantes innovaciones tecnológicas en su época y contribuyeron a disminuir el trabajo necesario para desarrollar programas informáticos (Backus, 1978).

Con el transcurso de las décadas, numerosas empresas desarrollaron sistemas informáticos cada vez más complejos, utilizando los lenguajes y las tecnologías disponibles en cada período histórico. Parte de estos sistemas continúa utilizándose en actividades industriales, financieras, científicas y empresariales, incluso cuando existen lenguajes de programación más recientes. En particular, las aplicaciones industriales especializadas, como los programas de diseño asistido por computadora (CAD), pueden incorporar componentes y algoritmos desarrollados y perfeccionados durante muchos años. La sustitución completa de estos sistemas no consiste simplemente en traducir su código a otro lenguaje y requerirá inversiones considerables en desarrollo, capacitación, pruebas, validación y mantenimientos (Jullien et al., 2002).

Por esta razón, muchas organizaciones prefieren conservar parte de su software existente, mantener personal capacitado en lenguajes antiguos y modernizar progresivamente determinados componentes, en lugar de reconstruir toda su infraestructura informática desde cero. Esta decisión puede prolongar la utilización de tecnologías cuyas herramientas de desarrollo, bibliotecas o arquitecturas dificultan la incorporación de nuevas funcionalidades (Jullien et al., 2002; Meyerovich & Rabkin, 2013).

Un ejemplo histórico de estas dificultades se encuentra en el desarrollo del software de diseño asistido por computador EUCLID, de Matra Datavision, cuya implementación original utilizaba principalmente Fortran. La acumulación de modificaciones y dependencias internas dificultó progresivamente su mantenimiento y evolución, hasta motivar el desarrollo de una nueva arquitectura de software basada en C++, denominada CAS.CADE, que incorporaba un motor de modelado geométrico tridimensional. Este proyecto requirió la participación de aproximadamente treinta desarrolladores y una inversión estimada de ocho millones de euros. El caso ilustra cómo las arquitecturas heredadas pueden incrementar la complejidad de incorporar nuevas funcionalidades y cómo su modernización puede exigir inversiones considerables (Jullien et al., 2002).

Consultar el informe original (PDF, 218 páginas) - revisa especialmente el capítulo 7.1, «L'exemple d'Open Cascade», páginas 111-113 (Jullien et al., 2002).

Una segunda razón que explica la coexistencia de numerosos lenguajes de programación es la influencia de la formación universitaria y la experiencia acumulada por distintas generaciones de desarrolladores. A lo largo de las décadas, determinados lenguajes se han consolidado como herramientas de enseñanza en universidades y centros de formación, dando lugar a comunidades de profesionales que adquirieron conocimientos y experiencia trabajando con ellos. Cuando estos profesionales ingresan al mercado laboral, tienden a utilizar los lenguajes que ya conocen para resolver nuevos problemas de ingeniería y desarrollar aplicaciones en ámbitos que no necesariamente fueron considerados durante el diseño original de dichas tecnologías (Gomes et al., 2026; Meyerovich & Rabkin, 2013).

De esta manera, la popularidad de un lenguaje puede extenderse hacia nuevas áreas de aplicación, incluso cuando existen alternativas que podrían ofrecer ventajas técnicas o una mayor productividad para determinadas tareas. Meyerovich y Rabkin (2013) demostraron empíricamente que la experiencia previa de los desarrolladores, la disponibilidad de bibliotecas y la existencia de código desarrollado anteriormente influyen considerablemente en la elección de un lenguaje de programación, mientras que sus características técnicas intrínsecas tienen una importancia relativamente menor.

Este fenómeno también responde a razones económicas y organizacionales. Cuando una empresa dispone de numerosos profesionales capacitados en un determinado lenguaje, adoptar otro puede requerir inversiones adicionales en formación, contratación de especialistas, adaptación de herramientas, desarrollo de nuevas librerías inexistentes, entre otros (Shrestha et al., 2020; Meyerovich & Rabkin, 2013).

La influencia de las instituciones educativas también contribuye a perpetuar este fenómeno. Gomes et al. (2026), en un estudio de 150 cursos de programación impartidos en 83 universidades, identificaron una concentración de la enseñanza introductoria en lenguajes ampliamente establecidos, particularmente Python, Java, C y C++. Esta concentración contribuye a mantener una oferta continua de desarrolladores familiarizados con estas tecnologías y facilita que las organizaciones continúen utilizándolas. En consecuencia, la permanencia de un lenguaje de programación no depende exclusivamente de su eficiencia computacional o de la facilidad con que permite desarrollar software, sino también de su difusión educativa, la experiencia de sus usuarios y

los costos económicos asociados a la adopción de tecnologías alternativas.

Una tercera razón que explica la existencia de numerosos lenguajes de programación es la necesidad de encontrar diferentes equilibrios entre la facilidad para desarrollar programas y la eficiencia con que estos utilizan los recursos computacionales. Los lenguajes de programación pueden clasificarse, entre otros criterios, según su nivel de abstracción. Los lenguajes de alto nivel, como Python, Java y JavaScript, permiten expresar operaciones mediante instrucciones relativamente comprensibles para las personas, ocultando numerosos detalles relacionados con el funcionamiento interno del computador. Por el contrario, los lenguajes de bajo nivel, como el lenguaje ensamblador, permiten trabajar con instrucciones directamente relacionadas con la arquitectura del procesador, pero generalmente requieren que el programador controle una mayor cantidad de detalles técnicos. En consecuencia, los lenguajes de alto nivel pueden reducir el tiempo y el esfuerzo necesarios para desarrollar determinadas aplicaciones, mientras que los lenguajes de menor nivel pueden facilitar un control más directo sobre la memoria y las operaciones ejecutadas por el hardware (Nisan & Schocken, 2005; Python Software Foundation, s. f.-d).

Sin embargo, independientemente del lenguaje utilizado para desarrollar un programa, las operaciones finalmente ejecutadas por el procesador corresponden a instrucciones de lenguaje máquina. Por esta razón, los programas escritos en lenguajes de alto nivel requieren mecanismos de compilación, interpretación o una combinación de ambos para poder ejecutarse. Por ejemplo, en una implementación habitual de Python, una operación aritmética puede involucrar comprobaciones de tipos de datos, manipulación de objetos y otras operaciones internas que no aparecen explícitamente en el código escrito por el desarrollador. Esto significa que una instrucción sencilla desde la perspectiva del programador puede requerir numerosas operaciones adicionales durante su ejecución. En determinados casos, estas operaciones introducen costos computacionales que podrían evitarse mediante implementaciones más especializadas con lenguajes de bajo nivel. No obstante, los compiladores modernos también pueden aplicar optimizaciones que producen código máquina altamente eficiente, por lo que un mayor nivel de abstracción no implica necesariamente una

menor eficiencia de ejecución (Nisan & Schocken, 2005; Python Software Foundation, s. f.-a).

Estas diferencias adquieren importancia cuando los programas deben procesar grandes cantidades de información o ejecutar repetidamente operaciones computacionales. En aplicaciones como simulaciones físicas, procesamiento de imágenes, videojuegos, sistemas de control y cálculos científicos, pequeñas diferencias en los recursos necesarios para realizar una operación pueden acumularse cuando esta se ejecutan de forma significativa. Por esta razón, la elección de un lenguaje de programación puede influir en el tiempo de ejecución, el consumo de memoria y la energía utilizada por el hardware. Sin embargo, estos resultados no dependen exclusivamente del lenguaje, sino también de los algoritmos empleados, la calidad de las implementaciones, las optimizaciones del compilador o intérprete y las características del computador utilizado (Pereira et al., 2017).

Por ende, para aumentar la productividad del desarrollador sin sacrificar rendimiento al generar el código en lenguaje de máquinas, personas innovar e inventar nuevos leguajes especializados determinados tipos de problemas computacionales. Estos lenguajes, denominados lenguajes de dominio específico, incorporan estructuras e instrucciones adaptadas a un conjunto particular de operaciones, permitiendo que los desarrolladores expresen determinados algoritmos con mayor facilidad y que sus compiladores aprovechen las características de estas operaciones para generar código ejecutable optimizado (Mernik et al., 2005).

Sin embargo, esta especialización implica renunciar a parte de la generalidad de los lenguajes convencionales, por lo que sus ventajas pueden disminuir considerablemente cuando se utilizan fuera del ámbito para el cual fueron diseñados (Mernik et al., 2005; Ragan-Kelley et al., 2013). Un ejemplo es Halide, un lenguaje desarrollado específicamente para el procesamiento de imágenes, cuyo compilador permite optimizar automáticamente aspectos relacionados con la paralelización, la vectorización y el uso de la memoria.

En determinadas pruebas experimentales, sus autores obtuvieron programas hasta cinco veces más rápidos que implementaciones equivalentes desarrolladas en C y optimizadas manualmente por expertos, utilizando código escrito en un tiempo considerablemente menor (Ragan-Kelley et al., 2013). Estos resultados

ilustran cómo el desarrollo de nuevos lenguajes puede responder simultáneamente a la necesidad de reducir el esfuerzo de programación y mejorar la eficiencia computacional en aplicaciones específicas.

Una cuarta razón que explica la creación de nuevos lenguajes de programación es la necesidad de mejorar la seguridad informática de los programas mediante mecanismos incorporados directamente en el diseño del lenguaje. Determinados lenguajes permiten realizar operaciones que, si no se programan correctamente, pueden generar vulnerabilidades aprovechables por terceros para acceder a información, modificar datos o ejecutar instrucciones no autorizadas. Por ejemplo, lenguajes como C y C++ permiten trabajar directamente con direcciones de memoria mediante punteros, lo que proporciona un control detallado sobre los recursos del computador, pero también permite cometer errores relacionados con el acceso a posiciones de memoria no autorizadas, la escritura fuera de los límites de un arreglo o la utilización de memoria previamente liberada. Estas vulnerabilidades han sido ampliamente estudiadas en seguridad informática y pueden comprometer el funcionamiento de sistemas completos (Szekeres et al., 2013).

Un ejemplo histórico de cómo las preocupaciones relacionadas con la seguridad informática pueden contribuir al reemplazo de determinadas tecnologías es el abandono de Adobe Flash en favor de tecnologías web abiertas como HTML5, JavaScript y CSS. Durante muchos años, Adobe Flash permitió desarrollar animaciones, videojuegos y aplicaciones interactivas mediante el lenguaje de programación ActionScript, cuyo código se ejecutaba utilizando Adobe Flash Player, un complemento que debía instalarse o encontrarse integrado en el navegador (Laforge, 2016; Adobe, 2021).

Sin embargo, las vulnerabilidades de seguridad asociadas a Flash Player, junto con problemas de rendimiento, consumo energético y compatibilidad, contribuyeron a que los principales navegadores comenzaran a favorecer tecnologías que permitieran ejecutar estas funcionalidades sin depender de dicho complemento (Laforge, 2016; Adobe, 2021).

Es importante distinguir que Flash Player no fue reemplazado exclusivamente

por HTML5, sino por un conjunto de tecnologías web que asumieron diferentes funciones. JavaScript pasó a desempeñar muchas de las funciones de programación interactiva que anteriormente se desarrollaban mediante ActionScript, mientras que HTML5 incorporó elementos para representar contenidos multimedia, como videos y gráficos mediante Canvas (Laforge, 2016; World Wide Web Consortium, 2014; WHATWG, 2026b).

Otras tecnologías, como CSS, SVG y WebGL, permitieron implementar animaciones y gráficos interactivos. En 2016, Google anunció que Chrome daría prioridad a HTML5 frente a Flash, destacando ventajas relacionadas con la seguridad, la eficiencia energética y los tiempos de carga (Google, 2016) (Laforge, 2016).

Finalmente, Adobe dejó de ofrecer soporte a Flash Player el 31 de diciembre de 2020 y comenzó a bloquear su ejecución el 12 de enero de 2021 (Adobe, 2021). Este caso demuestra cómo las exigencias de seguridad informática pueden contribuir al reemplazo de plataformas tecnológicas y a la adopción de alternativas que reduzcan la dependencia de componentes vulnerables, aunque no constituye estrictamente un ejemplo de creación de un nuevo lenguaje de programación por motivos de seguridad (Adobe, 2021).

Lenguajes de programación utilizados para desarrollar páginas web

El uso extendido de estas tecnologías se explica por la evolución de la Web: HTML permitió estructurar documentos y conectarlos mediante enlaces; posteriormente, CSS separó la presentación del contenido y facilitó mantener diseños coherentes en numerosas páginas. JavaScript se incorporó a los navegadores para añadir interactividad y se consolidó gracias a su disponibilidad directa en ellos, extendiéndose después al servidor con Node.js. Python ganó presencia en el backend por su facilidad de lectura y por frameworks como Django y Flask, que simplificaron la creación de aplicaciones web. C# se difundió mediante ASP.NET y el ecosistema de Microsoft, especialmente en organizaciones que ya utilizaban sus herramientas. TypeScript apareció más tarde para facilitar el mantenimiento de aplicaciones JavaScript cada vez más grandes, incorporando comprobación de tipos y conservando la compatibilidad con su ecosistema. Así, su adopción respondió a necesidades sucesivas de estructura, diseño, interactividad y organización del software; aunque se agrupan aquí como tecnologías web, HTML es un lenguaje de marcado y CSS es un lenguaje de hojas de estilo (CERN, s. f.; W3C, s. f.; Wirfs-Brock & Eich, 2020; Python Software Foundation, s. f.-d; Microsoft, 2022, s. f.-a).

- **JavaScript.** Lenguaje de programación creado por Brendan Eich en **1995**, en Netscape. Su propósito inicial fue incorporar comportamiento e interactividad a las páginas web. Tiene tipado dinámico, funciones de primera clase y permite programación orientada a objetos y funcional. Se utiliza para responder a acciones del usuario, actualizar contenido, validar formularios y comunicarse con servidores. También permite desarrollar servicios backend mediante Node.js y aplicaciones con herramientas como React Native. Es especialmente adecuado para interfaces interactivas y, con Node.js, servicios que gestionan numerosas operaciones de entrada y salida sin bloquear la ejecución. Los cálculos intensivos requieren organizar el trabajo para no bloquear la interfaz o el bucle de eventos (Wirfs-Brock & Eich, 2020; OpenJS Foundation, s. f.-b).

- **Python.** Lenguaje de programación desarrollado por Guido van Rossum desde **1989**, con su primera publicación en **1991**. Es de alto nivel, tiene tipado dinámico, gestión automática de memoria y una sintaxis que utiliza indentación para organizar los bloques de código. Su propósito es facilitar la escritura de programas legibles y reutilizables. Se utiliza en automatización, análisis de datos, inteligencia artificial, investigación científica y backend. Destaca por la rapidez de desarrollo y su ecosistema de bibliotecas. En cálculos numéricos puede alcanzar un buen rendimiento cuando delega el trabajo en bibliotecas optimizadas; los bucles ejecutados directamente por el intérprete no suelen ser su principal fortaleza en tareas intensivas de CPU (Van Rossum, s. f.; Python Software Foundation, s. f.-d).

- **CSS.** Lenguaje de hojas de estilo propuesto inicialmente por Håkon Wium Lie en **1994**. La primera recomendación CSS1 se publicó en **1996**. Su propósito es definir la presentación de documentos HTML: colores, tipografías, márgenes, distribución, adaptación a pantallas y animaciones. Es declarativo y utiliza selectores, cascada, especificidad y herencia para determinar los estilos aplicados. Resulta eficiente para mantener una apariencia coherente en muchas páginas mediante reglas reutilizables. Algunas animaciones, como las de transformación y opacidad, pueden ejecutarse en el compositor del navegador; otras propiedades pueden exigir recalcular la distribución o volver a pintar elementos. No es un lenguaje de programación de propósito general (W3C, s. f.; Google, s. f.-a).

- **HTML.** Lenguaje de marcado desarrollado por Tim Berners-Lee en el CERN alrededor de 1990, como parte de las tecnologías iniciales de la World Wide Web. Su propósito es estructurar y describir el significado del contenido mediante elementos como títulos, párrafos, enlaces, listas, imágenes, tablas y formularios. No es un lenguaje de programación: por sí solo no define algoritmos ni la lógica general de una aplicación. Es adecuado para documentos y contenido enlazado, y su estructura semántica facilita la interpretación por navegadores y tecnologías de asistencia. Para contenido estático, permite entregar una estructura lista para procesar sin necesitar JavaScript para crearla. CSS define su presentación y JavaScript añade comportamiento (CERN, s. f.; Microsoft, s. f.-s).

- **TypeScript.** Lenguaje de programación desarrollado por Microsoft, con Anders Hejlsberg como uno de sus principales diseñadores. Se presentó públicamente en 2012 y alcanzó su versión 1.0 en 2014. Amplía JavaScript con un sistema de tipos que permite detectar ciertos errores durante el desarrollo. Incluye inferencia de tipos, interfaces y tipos genéricos, lo que facilita describir los datos y las funciones de una aplicación. Se utiliza tanto en frontend, con herramientas como React y Next.js, como en backend mediante Node.js. Destaca en proyectos grandes porque facilita el mantenimiento, el autocompletado y la modificación del código. Sus tipos no validan automáticamente los datos externos ni aumentan por sí solos la velocidad de ejecución; para comprobar los datos recibidos de un servidor se necesitan validaciones adicionales (Microsoft, 2022; Turner, 2014; Microsoft, s. f.-y).

- **C#.** Lenguaje de programación desarrollado por Microsoft bajo la dirección de Anders Hejlsberg. Fue anunciado en 2000 y su versión 1.0 se publicó en 2002, junto con .NET Framework. Tiene tipado estático, gestión automática de memoria y admite programación orientada a objetos, genérica y funcional. Su propósito es facilitar el desarrollo de aplicaciones y servicios mediante la plataforma .NET. Se utiliza en backend con ASP.NET Core, aplicaciones de escritorio y videojuegos con Unity. También permite desarrollar interfaces web con Blazor. Destaca por sus herramientas de desarrollo, la detección de errores de tipos antes de ejecutar y el soporte para programación asíncrona y paralela. Su rendimiento depende de la implementación y

del entorno; la gestión automática de memoria simplifica el desarrollo, pero puede introducir pausas de recolección de basura (Microsoft, 2000, s. f.-z,s).

Tecnología	Propósito de uso actual	Creación	Uso principal en 2026	Alternativa más eficiente
JavaScript (Wirfs-Brock & Eich, 2020; International Digital Economy Academy, s. f.)	Desarrollar la lógica y la interactividad de aplicaciones web y servicios ejecutados en servidores.	1995	Interfaces web, aplicaciones interactivas y backend con Node.js.	**MoonBit**. Lenguaje compilado a WebAssembly que puede mejorar el rendimiento de cálculos concretos; no reemplaza las API del navegador (MoonBit, 2024).
Python (Van Rossum, s. f.; Nim Contributors, s. f.)	Automatizar tareas, analizar datos y desarrollar programas y servicios.	1989: inicio. 1991: publicación.	Inteligencia artificial, análisis de datos, automatización y backend.	**Nim**. Compila a código nativo y puede resultar más rápido en tareas intensivas en CPU; tiene un ecosistema menor (Torano, 2017).
C# (Microsoft, s. f.-z; Zig Software Foundation, s. f.)	Desarrollar aplicaciones y servicios con la plataforma .NET.	2000: anuncio. 2002: versión 1.0.	Servicios web con ASP.NET Core, aplicaciones empresariales, escritorio y videojuegos con Unity.	**Zig**. Útil para componentes nativos con control directo de memoria; puede reducir el costo de ejecución en tareas concretas, pero no sustituye el ecosistema .NET (Do, 2026; GlebOlsen, 2026).
TypeScript (Microsoft, 2022; Ginger Bill, s. f.)	Desarrollar aplicaciones basadas en JavaScript con tipos que permiten detectar errores durante el desarrollo.	2012: publicación. 2014: versión 1.0.	Interfaces con React y Next.js, aplicaciones web y backend con Node.js.	**Odin**. Genera ejecutables nativos y permite gestionar recursos directamente; puede ser útil en componentes de cálculo intensivo, pero no sustituye TypeScript en React y Next.js (MariuzM, s. f.).
HTML5 (World Wide Web Consortium, 2014; SixtyFPS GmbH, s. f.)	Estructurar documentos e interfaces web. No es un lenguaje de programación.	2004: trabajo inicial. 2014: recomendación W3C.	Estructura semántica de páginas y aplicaciones web modernas.	**Slint**. Herramienta para interfaces nativas ligeras; puede utilizar menos recursos, pero no sustituye HTML5 dentro del navegador (Slint, s. f.).

Fuente: elaboración propia con base en documentación histórica de JavaScript, Python, C#, TypeScript y HTML5 y de las alternativas citadas.

Tecnología	Ventajas frente a otras opciones	Desventajas frente a otras opciones	Uso en frontend o backend y por qué
JavaScript (OpenJS Foundation, s. f.-b; Microsoft, s. f.-y)	Funciona directamente en los navegadores. Frente a TypeScript, permite comenzar sin configurar la comprobación de tipos. Con Node.js puede utilizarse el mismo lenguaje en el cliente y el servidor.	Frente a TypeScript y C#, detecta menos errores de tipos antes de ejecutar. En Node.js, los cálculos largos en el hilo principal pueden retrasar otras solicitudes y requieren trabajo adicional para ejecutarse en paralelo.	Ambos. En frontend controla eventos y actualiza la interfaz del navegador. En backend, Node.js permite crear API y servicios, especialmente cuando predominan las operaciones de red y base de datos.
Python (Python Software Foundation, s. f.-d,s; Pereira et al., 2017)	Su sintaxis facilita escribir prototipos y automatizaciones con poco código. Frente a JavaScript y C#, ofrece un ecosistema especialmente amplio para análisis de datos, cálculo científico e inteligencia artificial.	Los cálculos escritos únicamente en Python suelen ser más lentos que sus equivalentes compilados en C#. A diferencia de JavaScript, no se ejecuta directamente como lenguaje habitual del navegador; necesita herramientas adicionales.	Principalmente backend. Flask y Django permiten procesar solicitudes, gestionar usuarios y acceder a bases de datos. También facilita integrar análisis de datos e inteligencia artificial en el servidor. Puede generar HTML para el frontend.
C# (Microsoft, s. f.-a,s; Pereira et al., 2017)	Su tipado estático detecta errores antes de ejecutar. Frente a Python, suele ofrecer mayor rendimiento en cálculos escritos en el propio lenguaje. .NET integra herramientas para API, concurrencia y aplicaciones empresariales.	Frente a Python, puede requerir más estructura y configuración para tareas pequeñas. Frente a JavaScript, no se ejecuta directamente en el navegador: para una interfaz interactiva con C# se utilizan soluciones como Blazor.	Principalmente backend; también frontend. ASP.NET Core permite desarrollar API y servicios. Blazor permite construir interfaces con C#, ejecutadas en el servidor o en el navegador mediante WebAssembly, según el modelo elegido.
TypeScript (Microsoft, s. f.-y)	Frente a JavaScript, detecta errores de tipos durante el desarrollo y facilita modificar proyectos grandes. Conserva el acceso al ecosistema JavaScript y mejora el autocompletado y la documentación del código.	Frente a JavaScript, añade configuración y aprendizaje del sistema de tipos. Sus tipos no comprueban por sí solos los datos recibidos de una API ni hacen que la aplicación se ejecute más rápido.	Ambos. En frontend se utiliza con React y Next.js para organizar interfaces con tipos. En backend se utiliza con Node.js para desarrollar API y servicios. Permite compartir definiciones de datos entre ambas partes.
HTML5 (WHATWG, 2026b; World Wide Web Consortium, 2014)	Es la base estándar de las páginas web y funciona directamente en los navegadores. Frente a dibujar toda la interfaz en un lienzo gráfico, sus elementos semánticos facilitan la accesibilidad y la interpretación del contenido.	Por sí solo no implementa lógica de negocio, acceso a bases de datos ni interactividad compleja. Necesita CSS para el diseño y otros lenguajes para el comportamiento. No es una alternativa directa a los lenguajes de programación.	Frontend. Define la estructura de textos, enlaces, formularios y otros elementos que muestra el navegador. El backend puede generar HTML, pero la lógica del servidor se implementa con otro lenguaje.

Nota: comparación orientativa según las características de cada tecnología. Las ventajas dependen del proyecto, las bibliotecas y la implementación. HTML5 es un lenguaje de marcado; complementa a los lenguajes de programación.

GitHub & GitHub Desktop para Desarrollo Web

GitHub

GitHub es una plataforma en línea destinada al desarrollo, almacenamiento y administración de proyectos de software. Su funcionamiento se basa en Git, un sistema de control de versiones que permite registrar y gestionar los cambios realizados en los archivos de un proyecto. GitHub amplía estas funcionalidades mediante el alojamiento de repositorios en la nube y la incorporación de herramientas que facilitan la colaboración entre desarrolladores, la revisión del código, la planificación de tareas y la automatización de procesos de desarrollo (GitHub, s. f.-i).

Asimismo, permite mantener repositorios públicos o privados y proporciona herramientas para implementar aplicaciones y publicar sitios web. De acuerdo con su documentación oficial, GitHub está diseñado para apoyar las distintas etapas del ciclo de vida del desarrollo de software, desde la planificación inicial hasta su implementación y mantenimiento (GitHub, s. f.-i).

GitHub fue creado por Tom Preston-Werner, Chris Wanstrath y PJ Hyett en Estados Unidos. Su desarrollo comenzó en 2007 como un proyecto impulsado por Preston-Werner y Wanstrath, al que posteriormente se incorporó Hyett. La plataforma fue lanzada públicamente el 10 de abril de 2008, con el propósito de facilitar el alojamiento de repositorios de código y la colaboración entre desarrolladores mediante el sistema de control de versiones Git (Chacon, 2009; Wanstrath, 2008) (Chacon, 2009; Wanstrath, 2008).

Las principales herramientas y funcionalidades de GitHub son: (GitHub, s. f.-e,s)

- **GitHub Repositories:** Permite almacenar, organizar y administrar repositorios de proyectos de software, públicos o privados.

- **Git Version Control:** Permite registrar y consultar el historial de modificaciones de los archivos mediante Git.

- **GitHub Branches:** Permite desarrollar distintas versiones o funcionalida-

des de un proyecto en ramas independientes.

- **GitHub Pull Requests:** Permite proponer, revisar y aprobar modificaciones antes de incorporarlas a una rama del repositorio.

- **GitHub Issues:** Permite registrar errores, organizar tareas y administrar solicitudes de nuevas funcionalidades.

- **GitHub Projects:** Permite planificar y supervisar el desarrollo de proyectos mediante tablas, tableros y otros mecanismos de seguimiento.

- **GitHub Discussions:** Proporciona espacios de discusión para realizar preguntas, intercambiar conocimientos y debatir propuestas relacionadas con un proyecto.

- **GitHub Actions:** Permite automatizar procesos de compilación, pruebas, verificación y despliegue de software.

- **GitHub Codespaces:** Proporciona entornos de desarrollo alojados en la nube que permiten programar sin configurar completamente un entorno local.

- **GitHub Pages:** Permite publicar y alojar sitios web estáticos directamente desde repositorios de GitHub.

- **GitHub Packages:** Permite almacenar, administrar y distribuir paquetes de software.

- **GitHub Releases:** Permite publicar versiones de un programa, acompañadas de notas de lanzamiento y archivos descargables.

- **GitHub Copilot:** Proporciona herramientas de inteligencia artificial para asistir en la escritura, comprensión, modificación y revisión del código fuente.

- **GitHub Code Security:** Proporciona herramientas para identificar y gestionar vulnerabilidades de seguridad presentes en el código fuente.

- **GitHub Secret Protection:** Proporciona herramientas para detectar y prevenir la exposición de determinados secretos, como credenciales y claves de acceso.

- **GitHub Dependabot:** Permite identificar dependencias vulnerables y proponer actualizaciones de los paquetes utilizados por un proyecto.

- **GitHub Code Search:** Permite buscar código, archivos y referencias dentro de los repositorios.

- **GitHub Wikis:** Permite crear y mantener documentación asociada a los proyectos de software.

- **GitHub Desktop:** Proporciona una aplicación de escritorio para administrar repositorios Git mediante una interfaz gráfica.

- **GitHub CLI:** Proporciona una interfaz de línea de comandos para interactuar con los servicios de GitHub desde una terminal.

- **GitHub Mobile:** Permite consultar notificaciones, revisar modificaciones y gestionar determinadas actividades desde dispositivos móviles.

- **GitHub Organizations:** Permite organizar equipos de desarrolladores, administrar repositorios compartidos y establecer permisos de acceso.

- **GitHub Notifications:** Permite recibir y gestionar notificaciones relacionadas con modificaciones, revisiones, discusiones y actividades de los repositorios.

- **GitHub APIs:** Proporciona interfaces de programación que permiten consultar información e integrar las funcionalidades de GitHub con aplicaciones externas.

- **GitHub Webhooks:** Permite enviar notificaciones automáticas a sistemas externos cuando ocurren determinados eventos en GitHub.

- **GitHub Apps:** Permite desarrollar e instalar aplicaciones que amplían o automatizan las funcionalidades de GitHub.

- **GitHub Marketplace:** Permite descubrir herramientas e integraciones que complementan las funcionalidades de GitHub.

- **GitHub Sponsors:** Proporciona mecanismos para financiar económicamente a desarrolladores y organizaciones que mantienen proyectos de código abierto.

- **GitHub Gist:** Permite almacenar y compartir fragmentos de código y otros archivos de texto mediante repositorios Git simplificados.

GitHub Pull Request GitHub Pull Requests es una funcionalidad de GitHub que permite proponer, revisar y gestionar modificaciones en el código fuente de un proyecto antes de incorporarlas a una rama de destino del repositorio (GitHub, s. f.-b).

Su funcionamiento se basa en la creación de solicitudes de incorporación de cambios, mediante las cuales los desarrolladores pueden presentar las modificaciones realizadas en una rama de trabajo para que sean examinadas por otros colaboradores (GitHub, s. f.-b).

Estas solicitudes permiten comparar versiones del código, incorporar comentarios, solicitar correcciones y aprobar las modificaciones propuestas. Adicionalmente, pueden integrarse con GitHub Actions para ejecutar pruebas automatizadas antes de incorporar los cambios (GitHub, s. f.-b,s).

Una vez completado el proceso de revisión y satisfechos los requisitos establecidos para el repositorio, las modificaciones pueden fusionarse con la rama de destino. De esta manera, GitHub Pull Requests facilita el desarrollo colaborativo y contribuye al control de calidad del software (GitHub, s. f.-b).

GitHub Actions GitHub Actions es un servicio de automatización integrado en GitHub que permite definir y ejecutar flujos de trabajo asociados al desarrollo de software (GitHub, s. f.-l).

Estos flujos de trabajo pueden activarse automáticamente cuando ocurren determinados eventos en un repositorio, como la incorporación de modificaciones al código fuente, o ejecutarse manualmente y mediante programaciones temporales (GitHub, s. f.-l).

Su funcionamiento se basa en archivos de configuración escritos en YAML, que establecen las instrucciones que deben ejecutarse en máquinas virtuales o en infraestructura administrada por el desarrollador (GitHub, s. f.-l).

Entre sus principales aplicaciones se encuentran la compilación de programas, la ejecución de pruebas automatizadas y el despliegue de aplicaciones (GitHub, s. f.-l).

En consecuencia, GitHub Actions permite automatizar distintas actividades del ciclo de vida del desarrollo de software, reduciendo la necesidad de ejecutar manualmente tareas repetitivas (GitHub, s. f.-l).

Según la documentación oficial de GitHub, los ejecutores hospedados por la plataforma que utilizan Linux y Windows funcionan sobre infraestructura de Microsoft Azure, mientras que los ejecutores con macOS utilizan una infraestructura propia de GitHub. Adicionalmente, GitHub Actions permite configurar ejecutores autohospedados, que pueden funcionar en computadores personales, servidores físicos o máquinas virtuales proporcionadas por otros servicios de computación en la nube (GitHub, s. f.-f,s).

Por consiguiente, la infraestructura utilizada para ejecutar los procesos depende del tipo de ejecutor seleccionado y de la configuración establecida por el desarrollador (GitHub, s. f.-f,s).

GitHub Desktop

GitHub Desktop es una aplicación de escritorio gratuita y de código abierto, desarrollada para facilitar la administración de repositorios Git mediante una interfaz gráfica (GitHub, s. f.-a).

A diferencia de GitHub, que proporciona una plataforma en línea para alojar y compartir proyectos de software, GitHub Desktop permite gestionar los repositorios almacenados localmente en un computador y sincronizarlos con repositorios remotos. Mediante esta aplicación, los desarrolladores pueden registrar modificaciones, consultar el historial de versiones, crear ramas de desarrollo y transferir cambios entre el computador y GitHub, sin necesidad de ejecutar directamente comandos de Git mediante una terminal (GitHub, s. f.-a).

En consecuencia, GitHub Desktop constituye una herramienta que facilita la interacción entre el sistema de control de versiones Git y la plataforma GitHub (GitHub, s. f.-a).

¿Cómo se relaciona GitHub con Microsoft Azure para desarrollo Web?

1. Integración de GitHub con Microsoft Azure para administrar y desplegar el código de una aplicación web GitHub puede conectarse con Microsoft Azure para facilitar el desarrollo, la actualización y el despliegue de aplicaciones web. Esta integración permite utilizar el código fuente almacenado en un repositorio de GitHub para publicar y actualizar una aplicación alojada en los servicios de Microsoft Azure (Microsoft, 2025b).

GitHub almacena y administra el código fuente del proyecto, mientras que Microsoft Azure proporciona la infraestructura necesaria para ejecutar y publicar la aplicación en Internet. Mediante herramientas como GitHub Actions, es posible automatizar los procesos de compilación, verificación y despliegue del software. De esta manera, cuando los desarrolladores incorporan modificaciones al repositorio, estas pueden desencadenar automáticamente la actualización de la aplicación alojada en Azure (Microsoft, 2025b).

Por ejemplo, si se utiliza Docker, GitHub Actions puede construir la imagen de una aplicación, publicarla en Azure Container Registry y actualizar el contenedor ejecutado en Azure Container Apps (Microsoft, s. f.-t).

Esta integración también puede utilizarse con repositorios privados de GitHub. Por consiguiente, Microsoft Azure puede recibir y desplegar el código fuente de una aplicación sin necesidad de que sus archivos estén disponibles públicamente. Para ello, es necesario establecer mecanismos de autenticación y autorización que permitan acceder al repositorio y ejecutar las operaciones de despliegue correspondientes (Microsoft, 2025b; GitHub, s. f.-k).

GitHub utiliza un sistema de control de acceso basado en permisos para proteger los repositorios privados. Este sistema permite que únicamente los usuarios, aplicaciones y servicios autorizados puedan consultar, descargar o modificar su contenido (GitHub, s. f.-k).

Para establecer conexiones automatizadas con repositorios privados pueden utilizarse mecanismos como GitHub Apps, tokens de acceso y claves SSH, dependiendo de las necesidades y la configuración de la integración. Estos mecanismos permiten autorizar operaciones sin compartir necesariamente la contraseña del

propietario del repositorio (GitHub, s. f.-h,s).

Un token de acceso es una credencial digital que permite a una aplicación o usuario autenticarse ante un servicio y realizar determinadas operaciones según los permisos concedidos (GitHub, s. f.-h).

Por ejemplo, un token de GitHub puede configurarse para permitir únicamente la lectura del contenido de un repositorio privado, sin conceder permisos para modificar sus archivos. GitHub permite crear tokens con permisos específicos y restringirlos a determinados repositorios (GitHub, s. f.-h).

Los tokens deben mantenerse protegidos, ya que su divulgación podría permitir que terceros utilicen los permisos asociados mientras las credenciales continúen siendo válidas (GitHub, s. f.-h).

Es importante distinguir las credenciales utilizadas para acceder a GitHub de aquellas empleadas para autorizar el despliegue en Microsoft Azure. Para esta última operación, GitHub Actions puede utilizar mecanismos de identidad federada mediante OpenID Connect, evitando la necesidad de almacenar determinadas credenciales de larga duración (Microsoft, 2025b).

Finalmente, una aplicación web debe mantener medidas adecuadas de ciberseguridad independientemente de que su código fuente se encuentre en un repositorio público o privado. La privacidad del repositorio limita el acceso a su contenido, pero no sustituye los mecanismos de autenticación, autorización, protección de credenciales y seguridad necesarios para el funcionamiento de la aplicación (OWASP Foundation, s. f.; Microsoft, 2023a).

2. Integración de GitHub con Microsoft Azure para autenticar a los usuarios de una página web Una segunda funcionalidad, completamente independiente de la integración utilizada para administrar y desplegar el código fuente, consiste en utilizar GitHub como proveedor de autenticación para los usuarios de una aplicación web alojada en Microsoft Azure (Microsoft, 2026b).

Mediante el servicio Azure App Service, es posible implementar un sistema de inicio de sesión que permita a los visitantes de una página web identificarse utilizando sus cuentas personales de GitHub, como alternativa al inicio de sesión mediante cuentas de Microsoft o Google. Esta funcionalidad permite que los usuarios accedan a los servicios ofrecidos por la aplicación sin necesidad de crear una contraseña independiente para dicha página web (Microsoft, 2026b).

Este procedimiento utiliza el protocolo OAuth 2.0 para delegar la autenticación en GitHub. Cuando un usuario selecciona la opción de iniciar sesión con GitHub, es redirigido a esta plataforma para identificarse y autorizar la aplicación. Una vez completado el procedimiento, Azure App Service puede establecer una sesión autenticada para el usuario, sin que la página web necesite almacenar directamente su contraseña de GitHub (Microsoft, 2026b; GitHub, s. f.-c).

Para habilitar esta funcionalidad, el desarrollador debe registrar una aplicación OAuth en GitHub y configurar las credenciales correspondientes en el sistema de autenticación de Azure App Service. De este modo, GitHub actúa como proveedor externo de autenticación, mientras que Azure administra la integración con la aplicación web (Microsoft, 2026b).

Es importante señalar que este sistema de autenticación no concede a los usuarios acceso al repositorio donde se almacena el código fuente de la aplicación, ni les permite modificarlo. Su finalidad es exclusivamente verificar su identidad para que puedan utilizar los servicios ofrecidos por la página web. Los permisos que cada usuario tenga dentro de la aplicación deben administrarse mediante mecanismos de autorización independientes (Microsoft, 2026b; GitHub, s. f.-c).

Por consiguiente, una aplicación web puede utilizar GitHub como proveedor de autenticación aunque su código fuente no se encuentre almacenado en GitHub.

Asimismo, puede utilizar un repositorio privado de GitHub para desplegar sus actualizaciones en Microsoft Azure sin necesidad de ofrecer a sus usuarios un sistema de inicio de sesión mediante GitHub. Se trata, por tanto, de dos integraciones tecnológicas diferentes que cumplen funciones independientes (Microsoft, 2025b, 2026b).

Microsoft Azure

¿Que es Microsoft Azure?

Microsoft Azure es una plataforma de computación en la nube que proporciona servicios de procesamiento, almacenamiento, redes, bases de datos, análisis e inteligencia artificial. Permite a los desarrolladores y organizaciones crear, implementar, ejecutar y administrar aplicaciones mediante una infraestructura de servidores distribuidos globalmente, sin necesidad de mantener toda la infraestructura física de manera local (Microsoft, s. f.-u).

Fue presentada originalmente por Microsoft el 27 de octubre de 2008 bajo el nombre de Windows Azure, cuya disponibilidad comercial comenzó en febrero de 2010 (Microsoft, 2008, 2010).

La infraestructura de software de Microsoft Azure comprende diversos componentes que sustentan la ejecución y coordinación de sus servicios en la nube. Estos componentes se implementan mediante tecnologías seleccionadas de acuerdo con los requisitos técnicos de cada servicio, por lo que su desarrollo no corresponde a un único lenguaje de programación. Entre las tecnologías presentes en sus componentes públicos se encuentran el lenguaje C# y la plataforma .NET, utilizados para implementar parte de los entornos de ejecución que permiten el funcionamiento de los servicios (Microsoft, s. f.-e).

La infraestructura de Microsoft Azure se encuentra distribuida geográficamente en centros de datos (data centers) que contienen servidores físicos, sistemas de almacenamiento, de refrigeración, seguridad, infraestructura eléctrica y equipos de comunicación. Estos centros de datos se organizan en regiones geográficas interconectadas mediante redes de alta capacidad, lo que permite proporcionar servicios de computación, almacenamiento y procesamien-

to de información a través de Internet (Microsoft, s. f.-x,s).

No todos los datos de Azure se guardan en un único lugar. La región donde se almacenan y las copias adicionales que puedan existir dependen del servicio utilizado, su configuración y las políticas de residencia de datos aplicables (Microsoft, s. f.-x).

Esta permite que múltiples usuarios y organizaciones utilicen recursos informáticos sin necesidad de adquirir, instalar o administrar directamente los equipos físicos que los proporcionan (Microsoft, s. f.-u).

Para comprender cómo funciona, es importante distinguir tres elementos: la infraestructura física, el software que permite operar Azure y el código fuente de las aplicaciones que desarrollan sus clientes (Microsoft, s. f.-h,s, 2026e).

Los servicios son desarrollados y mantenidos principalmente por equipos de ingeniería de Microsoft, que incluyen desarrolladores de software, especialistas en infraestructura, bases de datos, redes, ciberseguridad y operaciones (Microsoft, s. f.-h; Czerwonka et al., 2013).

Esto no significa que todo el software de Azure sea desarrollado exclusivamente por Microsoft. Algunos componentes utilizan proyectos de código abierto, bibliotecas externas y tecnologías mantenidas por comunidades de desarrolladores (Microsoft, s. f.-e,s).

Microsoft mantiene gran parte del código de sus servicios e infraestructura en sistemas de desarrollo y repositorios internos con acceso restringido. La ubicación y organización exactas de todos esos repositorios no son públicas (Czerwonka et al., 2013).

Microsoft Azure utiliza diferentes sistemas operativos en su infraestructura de computación en la nube. Según Microsoft, parte de su infraestructura de virtualización se basa en versiones personalizadas y reforzadas de Windows Server, junto con tecnologías de hipervirtualización que permiten ejecutar múltiples máquinas virtuales sobre servidores físicos (Microsoft, s. f.-h).

Windows Server

¿Qué es Windows Server?

Windows Server es una familia de sistemas operativos desarrollada por Microsoft, diseñada para administrar infraestructuras informáticas, ejecutar aplicaciones empresariales y proporcionar servicios de red, almacenamiento y virtualización. A diferencia de los sistemas operativos orientados al usuario final, Windows Server incorpora herramientas especializadas para administrar recursos computacionales, usuarios y servicios en entornos empresariales y de computación en la nube (Microsoft, s. f.-w).

Asimismo, Microsoft utiliza tecnologías basadas en Linux, incluyendo Azure Linux, una distribución optimizada para determinadas cargas de trabajo y ejecución de contenedores. Esta arquitectura permite utilizar diferentes sistemas operativos según las necesidades de los servicios y aplicaciones, sin limitar la plataforma exclusivamente a Windows o Linux (Microsoft, s. f.-v).

Característica	Linux para servidores	Windows Server	Consecuencia
Licencias y modelo de desarrollo	Familia de sistemas basada en un núcleo de código abierto. Las distribuciones pueden ser comunitarias o contar con soporte comercial. (Debian Project, 2022)	Sistema operativo comercial desarrollado por Microsoft, disponible mediante distintas ediciones y modalidades de licencia. (Microsoft, s. f.-ag)	En Windows, el costo depende de núcleos, usuarios y licencias para virtualización. Linux puede evitar licencias comerciales, pero no necesariamente los gastos de soporte. (Debian Project, 2022; Microsoft, s. f.-ag)
Instalación y recursos	Permite instalaciones mínimas, sin escritorio gráfico, a las que se incorporan los componentes que requiere cada servicio. (Canonical, s. f.-a)	Puede instalarse con interfaz gráfica o en modalidad reducida, denominada *Server Core*. (Microsoft, 2025d)	Una instalación mínima reduce componentes que mantener y posibles puntos de exposición, aunque ciertas funciones pueden quedar fuera. (Microsoft, 2025d)
Administración	Suele administrarse mediante terminal, acceso remoto, archivos de configuración y automatización; la interfaz gráfica es opcional. (Canonical, s. f.-c)	Permite administración gráfica, por línea de comandos y de forma remota mediante herramientas integradas. (Microsoft, 2025d)	El acceso remoto facilita operar varios servidores sin desplazarse; exige configurar conexiones seguras y dominar las herramientas de administración. (Canonical, s. f.-c; Microsoft, 2025d)
Aplicaciones y backend	Ejecuta servicios web y aplicaciones de servidor desarrolladas en diversos lenguajes y entornos de ejecución. (Canonical, s. f.-e)	También ejecuta aplicaciones web y servicios, incluidos aquellos que requieren tecnologías específicas de Windows. (Microsoft, s. f.-m, 2023b)	Los servicios multiplataforma pueden desplegarse en ambos sistemas; las dependencias exclusivas de Windows, como *.NET Framework*, pueden dificultar la migración. (Microsoft, s. f.-o, 2023b)
Contenedores	Es el entorno habitual de los contenedores Linux, que aíslan procesos y utilizan funcionalidades del núcleo Linux. (Docker, s. f.-e,s)	Permite ejecutar contenedores Windows para aplicaciones dependientes de su entorno de sistema. (Microsoft, s. f.-ae)	Las imágenes deben ser compatibles con el sistema del contenedor (Docker, s. f.-e; Microsoft, s. f.-ae). En AKS, las aplicaciones Windows requieren nodos de usuario Windows (Microsoft, s. f.-r).
Ciberseguridad	Dispone de separación de usuarios, permisos y mecanismos de protección configurables según la distribución. (Canonical, s. f.-d)	Ofrece autenticación, control de acceso y directivas de seguridad integradas en su ecosistema. (Microsoft, s. f.-ad)	Reducir servicios y privilegios limita la exposición, pero ambos sistemas requieren parches, control de accesos y configuraciones seguras. (Canonical, s. f.-d; Microsoft, 2025d)
Identidades y usuarios	Administra usuarios y grupos locales, y puede integrarse con servicios de identidad y directorios externos. (Canonical, s. f.-d,s)	Incluye Active Directory Domain Services para centralizar identidades y recursos de una organización. (Microsoft, s. f.-b)	Un directorio central evita duplicar cuentas de usuario. Linux puede integrarse con Active Directory mediante servicios de conexión adicionales. (Canonical, s. f.-b; Microsoft, s. f.-b)
Virtualización	Puede actuar como anfitrión de máquinas virtuales mediante tecnologías como KVM. (Linux Kernel Development Community, s. f.)	Integra Hyper-V para crear y operar máquinas virtuales sobre infraestructura compatible. (Microsoft, s. f.-l)	Ambos consolidan máquinas virtuales en un solo equipo (Linux Kernel Development Community, s. f.; Microsoft, s. f.-l). En Windows, la edición licenciada condiciona los derechos de virtualización (Microsoft, s. f.-ag).
Uso en Microsoft Azure	Puede utilizarse como sistema operativo de máquinas virtuales y nodos de contenedores en Azure. (Microsoft, s. f.-ab,s)	También está disponible para máquinas virtuales y determinados nodos de contenedores en Azure. (Microsoft, s. f.-ab,s)	Ciertas imágenes Linux agregan costos de soporte (Microsoft, s. f.-q); en Windows, *Azure Hybrid Benefit* puede reducir el gasto con licencias elegibles (Microsoft, s. f.-f).

Fuente: elaboración propia a partir de la documentación oficial de Debian, Canonical, Docker, el núcleo Linux y Microsoft. Las consecuencias se fundamentan en las fuentes citadas por fila. Comparación cualitativa, no una medición de rendimiento.

¿Qué tecnologías de programación se utilizan en la infraestructura de Microsoft Azure y que factores explican su adopción?

La infraestructura de software de Microsoft Azure no utiliza un único lenguaje de programación, ya que la plataforma está compuesta por numerosos servicios, entornos de ejecución y componentes de infraestructura que cumplen funciones diferentes. Por esta razón, no sería técnicamente correcto afirmar que todo Azure está desarrollado en C# o en cualquier otro lenguaje. Sin embargo, C# y .NET son tecnologías importantes en determinados componentes de su funcionamiento interno. Un ejemplo verificable es Azure Functions Host, el entorno de ejecución de Azure Functions, cuyo repositorio público contiene código fuente desarrollado en C# sobre .NET (Microsoft, s. f.-e).

Para comprender el origen de esta relación tecnológica, es necesario considerar los antecedentes de la plataforma. En octubre de 2008, Microsoft presentó Windows Azure, denominación original de Microsoft Azure, como una plataforma de servicios en la nube destinada a facilitar la creación, el alojamiento y la ejecución de aplicaciones. Desde su presentación, Microsoft destacó que los desarrolladores podían aprovechar conocimientos y herramientas que ya utilizaban, especialmente .NET Framework y Visual Studio, sin limitarse exclusivamente a estas tecnologías (Microsoft, 2008).

En las primeras etapas de Windows Azure, C# se empleaba frecuentemente para desarrollar la lógica de las aplicaciones, mientras que .NET Framework proporcionaba bibliotecas, servicios y un entorno de ejecución para el software escrito sobre esta plataforma. Visual Studio, por su parte, permitía escribir, organizar, compilar y depurar el código, además de configurar y preparar los paquetes destinados a su implementación en la nube. Así, los desarrolladores familiarizados con C#, ASP.NET y las herramientas de Microsoft podían trasladar parte de su experiencia al desarrollo de aplicaciones alojadas en Windows Azure (Nakashima et al., 2010) (Nakashima et al., 2010).

Esta integración representó una ventaja para los desarrolladores familiarizados con el ecosistema de Microsoft, pues podían reutilizar conocimientos, bibliotecas y herramientas en lugar de aprender desde cero un entorno de desarrollo completamente diferente. No obstante, trasladar una aplicación a la nube no siempre equivalía a copiarla sin modificaciones: en ciertos casos era necesario adaptar el manejo del almacenamiento, la comunicación entre compo-

nentes y la arquitectura de procesamiento para aprovechar correctamente la escalabilidad y la distribución de recursos de Windows Azure (Microsoft, 2008; Nakashima et al., 2010).

Con el desarrollo posterior de la plataforma, Microsoft Azure incorporó servicios y entornos de ejecución compatibles con diferentes tecnologías. Por ejemplo, Azure Functions admite el desarrollo de funciones mediante C#, Java, JavaScript, TypeScript, Python y PowerShell, entre otros mecanismos de extensibilidad; en el caso de TypeScript, su ejecución se realiza mediante su conversión a JavaScript. Esta compatibilidad indica qué lenguajes pueden utilizar los desarrolladores para crear aplicaciones en el servicio, pero **no permite deducir que todos esos lenguajes se utilicen para programar la infraestructu interna de Azure** (Microsoft, 2026d).

Servicios de Microsoft Azure útiles para desarrollo Web

App Service

¿Qué es App Service?

Azure App Service es un servicio de plataforma como servicio completamente administrado, que permite alojar y ejecutar aplicaciones web, servicios de backend para aplicaciones móviles y API RESTful en la nube, sin necesidad de administrar directamente la infraestructura de servidores. Admite tecnologías como .NET, Java, Node.js, Python y PHP, tanto en entornos Windows como Linux, e incorpora funcionalidades de escalabilidad, seguridad y despliegue continuo (Microsoft, 2026e).

Azure Cosmos DB en Micrsofot Azure

¿Que es Azure Cosmos DB en Micrsofot Azure?

Azure Cosmos DB es un servicio de base de datos en la nube de Microsoft Azure, totalmente administrado y distribuido globalmente, que permite almacenar, consultar y gestionar información mediante distintos modelos de datos y API. Está diseñado para proporcionar alta disponibilidad, baja latencia y escalabilidad en aplicaciones modernas (Microsoft, 2026a).

Una API (Application Programming Interface, o interfaz de programación de aplicaciones) aplicada a Azure Cosmos DB es un mecanismo de comunicación que permite a una aplicación acceder a una base de datos alojada en Microsoft Azure para almacenar, consultar, modificar o eliminar información (Microsoft, 2026a, s. f.-d).

Azure Cosmos DB almacena datos de aplicaciones y permite acceder a ellos mediante diferentes API (interfaces de programación de aplicaciones) (Microsoft, 2026a).

Azure Container Registry

¿Qué es Azure Container Registry?

Azure Container Registry es un servicio administrado de Microsoft Azure que permite almacenar, gestionar y distribuir imágenes de contenedores y artefactos relacionados mediante registros privados. Facilita la integración con herramientas de desarrollo, automatización e implementación de aplicaciones basadas en contenedores, proporcionando mecanismos de autenticación, control de acceso y distribución de imágenes (Microsoft, s. f.-n).

Una imagen de contenedor es un paquete de software que contiene todo lo necesario para ejecutar una aplicación, incluyendo su código, librerías, dependencias, configuraciones y, cuando corresponde, componentes del sistema operativo (Docker, s. f.-d).

En lugar de instalar y configurar todos estos componentes manualmente en cada servidor, puedes empaquetarlos en una imagen de Docker (Docker, s. f.-d).

Docker es una plataforma de software de código abierto que permite desarrollar, empaquetar, distribuir y ejecutar aplicaciones dentro de entornos aislados llamados contenedores (Docker, s. f.-d,s).

Su principal objetivo es garantizar que una aplicación pueda ejecutarse de manera consistente en diferentes equipos o servidores, sin necesidad de instalar manualmente todas sus dependencias (Docker, s. f.-d).

La ausencia de contenedores como Docker puede incrementar los riesgos asociados con la compatibilidad, estabilidad y seguridad de las aplicaciones, especialmente cuando estas utilizan dependencias externas que se actualizan automáticamente. Si una actualización modifica o elimina funcionalidades previamente utilizadas, la aplicación puede presentar errores de ejecución, incompatibilidades o interrupciones del servicio (Docker, s. f.-a).

Desde la perspectiva de la ciberseguridad, la incorporación automática de dependencias externas también representa un riesgo, debido a la posibilidad de que una actualización introduzca código malicioso o vulnerabilidades. Esto

podría comprometer la integridad de la aplicación, la confidencialidad de la información y la seguridad del sistema donde se ejecuta (OWASP Foundation, s. f.; Docker, s. f.-a).

Docker permite reducir estos riesgos mediante la creación de imágenes de contenedores con versiones específicas de las dependencias y configuraciones previamente definidas. Esto favorece la reproducibilidad del entorno de ejecución y evita que las actualizaciones externas modifiquen automáticamente las dependencias incluidas en una imagen ya construida. Sin embargo, Docker no garantiza por sí mismo la seguridad, ya que las imágenes también pueden incorporar componentes vulnerables o maliciosos. Por ello, es necesario complementar su utilización con mecanismos de verificación, control de versiones y actualización segura de dependencias (Docker, s. f.-a).

¿Que son los Roles en Azure Container Registry?

Los roles en Azure Container Registry (ACR) son conjuntos de permisos que determinan qué operaciones pueden realizar los usuarios, aplicaciones o servicios sobre un registro de contenedores (Microsoft, s. f.-c).

Estos roles se administran mediante el control de acceso basado en roles (RBAC, Role-Based Access Control) de Azure y permiten restringir acciones como consultar, descargar, publicar, modificar o eliminar imágenes de contenedores (Microsoft, s. f.-c).

Los roles permiten aplicar el principio de mínimos privilegios, que consiste en conceder a cada usuario o servicio únicamente los permisos necesarios para realizar sus funciones (Microsoft, s. f.-c; OWASP Foundation, s. f.).

Por ejemplo, una aplicación que necesita descargar una imagen para ejecutarse no debería tener permisos para modificarla o eliminarla. De esta manera, si las credenciales de dicha aplicación se ven comprometidas, se limita el alcance de las operaciones que un atacante podría realizar (Microsoft, s. f.-c).

Además, mediante el control de acceso basado en atributos (ABAC), Azure Container Registry puede restringir determinados permisos a repositorios específicos, evitando que una identidad tenga acceso innecesario a todas las imágenes almacenadas (Microsoft, s. f.-c).

Tecnología que podemos desarrollar con Microsoft Azure

- **Cloud-Hosted Web API**: Interfaz de programación de aplicaciones que expone operaciones y recursos de un servicio ejecutado en una infraestructura de computación en la nube. Permite que aplicaciones cliente envíen solicitudes mediante HTTP o HTTPS para transmitir datos, consultar información o solicitar operaciones, y reciban respuestas con los resultados y el estado de su ejecución. El intercambio puede realizarse mediante formatos como JSON, sin que el cliente necesite conocer la implementación interna del servicio. En Microsoft Azure, estas API pueden desarrollarse con lenguajes como Python y alojarse mediante Azure App Service o implementarse mediante funciones activadas por solicitudes HTTP en Azure Functions. De este modo, permiten delegar cálculos, procesamiento de información y acceso a datos en servicios remotos, separando estas responsabilidades de la aplicación cliente (Microsoft, 2025c, 2026e,f, s. f.-j).

- **Aplicación Web**: Sistema de software que permite a los usuarios acceder a funcionalidades e interactuar con información mediante una interfaz web. Su funcionamiento puede distribuirse entre una interfaz que recibe las solicitudes del usuario y componentes del servidor que procesan operaciones, se comunican con servicios externos y consultan o modifican bases de datos. Según las necesidades de la aplicación, también puede incorporar mecanismos de autenticación, almacenamiento temporal y procesamiento de tareas en segundo plano. En Microsoft Azure, estas aplicaciones pueden alojarse mediante servicios como Azure App Service, que permite ejecutar aplicaciones desarrolladas con tecnologías como .NET, Java, Python y Node.js sin administrar directamente la infraestructura subyacente (Microsoft, 2026e,c).

- **Aplicación Móvil**: Sistema de software desarrollado para utilizarse en dispositivos móviles, como teléfonos inteligentes y tabletas, que permite realizar tareas y acceder a servicios desde plataformas como Android e iOS. Su funcionamiento puede integrar componentes ejecutados en el dispositivo con servicios de procesamiento y almacenamiento alojados en la nube. Esta integración permite incorporar funcionalidades como la autenticación de usua-

rios, el almacenamiento de información, la sincronización de datos para su consulta sin conexión y el envío de notificaciones. De esta manera, la aplicación combina la interacción del usuario desde el dispositivo móvil con servicios remotos que amplían sus capacidades (Microsoft, 2026e; Ashley, 2015; Microsoft, s. f.-ac).

Visualización Web vs. Renderizado Nativo en el Sistema Operativo

Las tecnologías utilizadas para construir interfaces gráficas permiten integrar la visualización y la interacción de las aplicaciones con cada sistema operativo. La siguiente tabla presenta las principales herramientas disponibles en Windows, macOS, Linux y Android, junto con resultados de estudios que comparan el consumo de recursos de aplicaciones nativas, web y basadas en Electron (Microsoft, s. f.-k; Apple, s. f.-b; The GNOME Project, s. f.; The Qt Company, s. f.-b; Google, s. f.-c).

Estos datos aportan contexto sobre el desempeño de las aplicaciones completas, pero no permiten evaluar de manera aislada la eficiencia del sistema de visualización ni establecer una superioridad general en fluidez o calidad visual. En particular, se resumen las mediciones disponibles de memoria RAM, utilización de CPU y consumo energético, y se indican los aspectos que no fueron evaluados en los estudios consultados. Las diferencias observadas deben interpretarse según las aplicaciones, los dispositivos y las condiciones de cada comparación, sin extrapolarlas a todas las herramientas de una plataforma (Horn et al., 2023; Metri et al., 2015; Milichiewicz & Badurowicz, 2025).

Sistema operativo	Lenguajes y herramientas nativas	Comparación conceptual	Memoria RAM	Procesador (CPU)	Consumo energético	Fluidez y animaciones	Calidad visual
Windows	C#, C++. WinUI, WPF, Windows Forms y Win32 (Microsoft, s. f.-af).	Las aplicaciones de escritorio pueden utilizar menos recursos que determinadas alternativas basadas en tecnologías web. Sin embargo, el resultado depende de la arquitectura y de las herramientas utilizadas (Metri et al., 2015; Milichiewicz & Badurowicz, 2025).	Las aplicaciones desarrolladas con Qt/Python presentaron un consumo de memoria inferior al de Electron. Esto no demuestra que todas las aplicaciones nativas de Windows consuman menos RAM que las aplicaciones web (Milichiewicz & Badurowicz, 2025).	No existe una ventaja general demostrada. La utilización del procesador depende de las operaciones ejecutadas y de la implementación de cada aplicación (Metri et al., 2015; Milichiewicz & Badurowicz, 2025).	Determinadas aplicaciones instaladas presentaron mayor eficiencia energética que sus equivalentes web. La ventaja no puede generalizarse a todas las aplicaciones (Metri et al., 2015).	No se demostró una superioridad general en fluidez, velocidad de animación o frecuencia de imágenes (Metri et al., 2015; Milichiewicz & Badurowicz, 2025).	No se demostró una superioridad general en nitidez, calidad gráfica o diseño visual (Metri et al., 2015; Milichiewicz & Badurowicz, 2025).
macOS	Swift, Objective-C. SwiftUI y AppKit (Apple, s. f.-c,s).	Las diferencias de rendimiento entre aplicaciones de escritorio dependen del framework utilizado. La comparación disponible enfrenta Qt/Python con Electron, no SwiftUI directamente con Safari (Milichiewicz & Badurowicz, 2025).	Qt/Python utilizó menos memoria RAM que Electron. Por tanto, algunas arquitecturas de escritorio pueden disminuir la demanda de memoria (Milichiewicz & Badurowicz, 2025).	No se estableció una ventaja general de las aplicaciones nativas sobre las web en utilización de CPU (Milichiewicz & Badurowicz, 2025).	No se estableció una conclusión comparativa sobre el consumo energético entre aplicaciones nativas de macOS y aplicaciones web (Milichiewicz & Badurowicz, 2025).	No se demostró una ventaja general de SwiftUI o AppKit en fluidez o animaciones frente a interfaces web (Milichiewicz & Badurowicz, 2025).	La investigación no estableció una superioridad estética o de nitidez de las interfaces nativas (Milichiewicz & Badurowicz, 2025).
Linux	C, C++, Python, entre otros. GTK y Qt (GTK Development Team, s. f.; The Qt Company, s. f.-a).	Las diferencias de consumo de recursos dependen de la tecnología utilizada para construir la interfaz. Las comparaciones entre Qt/Python y Electron muestran diferencias importantes en memoria (Milichiewicz & Badurowicz, 2025).	Qt/Python presentó un consumo de memoria inferior al de Electron. La ventaja corresponde a los frameworks comparados y no puede extenderse a todas las aplicaciones de Linux (Milichiewicz & Badurowicz, 2025).	No se estableció una reducción general de CPU atribuible al uso de interfaces nativas. Depende de la implementación y de las tareas ejecutadas (Milichiewicz & Badurowicz, 2025).	No se estableció una ventaja energética general de GTK o Qt frente a aplicaciones ejecutadas en navegadores web (Milichiewicz & Badurowicz, 2025).	No se demostró una mayor fluidez general de las interfaces nativas frente a las web (Milichiewicz & Badurowicz, 2025).	No se demostró una superioridad general en calidad gráfica. La apariencia depende del entorno de escritorio y de las herramientas utilizadas (Milichiewicz & Badurowicz, 2025).
Android	Kotlin, Java. Jetpack Compose y Android Views (Google, s. f.-b).	Las aplicaciones nativas mostraron una utilización más eficiente de los recursos computacionales que sus versiones web ejecutadas en Chrome, para las aplicaciones analizadas (Horn et al., 2023).	Las aplicaciones nativas utilizaron significativamente menos memoria RAM. Las versiones web incorporan la sobrecarga del navegador, que contribuye a un mayor consumo de memoria (Horn et al., 2023).	Las aplicaciones nativas presentaron una menor utilización del procesador en la comparación realizada. Esto puede disminuir la carga computacional del dispositivo (Horn et al., 2023).	Las aplicaciones nativas presentaron un consumo energético significativamente menor que sus versiones web. Esto puede favorecer la duración de la batería (Horn et al., 2023).	No se encontró evidencia suficiente para afirmar que las aplicaciones nativas presentaran mayor fluidez o mejores tiempos de cuadro (Horn et al., 2023).	No se evaluó una superioridad de calidad visual. El menor consumo de recursos no demuestra una mayor nitidez ni un mejor diseño (Horn et al., 2023).

Fuente: Elaboración propia a partir de (Microsoft, s. f.-af; Apple, s. f.-c,s; GTK Development Team, s. f.; The Qt Company, s. f.-a; Google, s. f.-b; Metri et al., 2015; Milichiewicz & Badurowicz, 2025; Horn et al., 2023).

¿Si es más eficiente utilizar los sistemas de visualización nativos de cada sistema operativo, entonces por que no se prefiere por sobre el sistema de visualización de nos navegadores con JavaScript, HTML y CSS?

Una de las principales razones por las que se utilizan HTML, CSS y JavaScript para desarrollar interfaces web, en lugar de recurrir directamente a los sistemas de visualización nativos de cada sistema operativo, es la posibilidad de utilizar una misma base de código en diferentes plataformas. Si cada página web tuviera que desarrollarse utilizando exclusivamente las tecnologías nativas de cada sistema operativo, sus desarrolladores necesitarían implementar y mantener versiones específicas para Windows, macOS, Linux, Android e iOS, lo que incrementaría considerablemente el trabajo de programación y mantenimiento. En cambio, mediante HTML, CSS y JavaScript, los desarrolladores pueden definir una única implementación de la interfaz, mientras que los navegadores se encargan de interpretar estas tecnologías y utilizar los mecanismos de visualización disponibles en cada sistema operativo. De esta manera, aunque pueden requerirse adaptaciones para garantizar la compatibilidad entre navegadores, las tecnologías web permiten reducir la duplicación del trabajo de desarrollo y facilitan la distribución de aplicaciones en múltiples plataformas (MDN Web Docs, s. f.-c).

Descripción general de la aplicación web

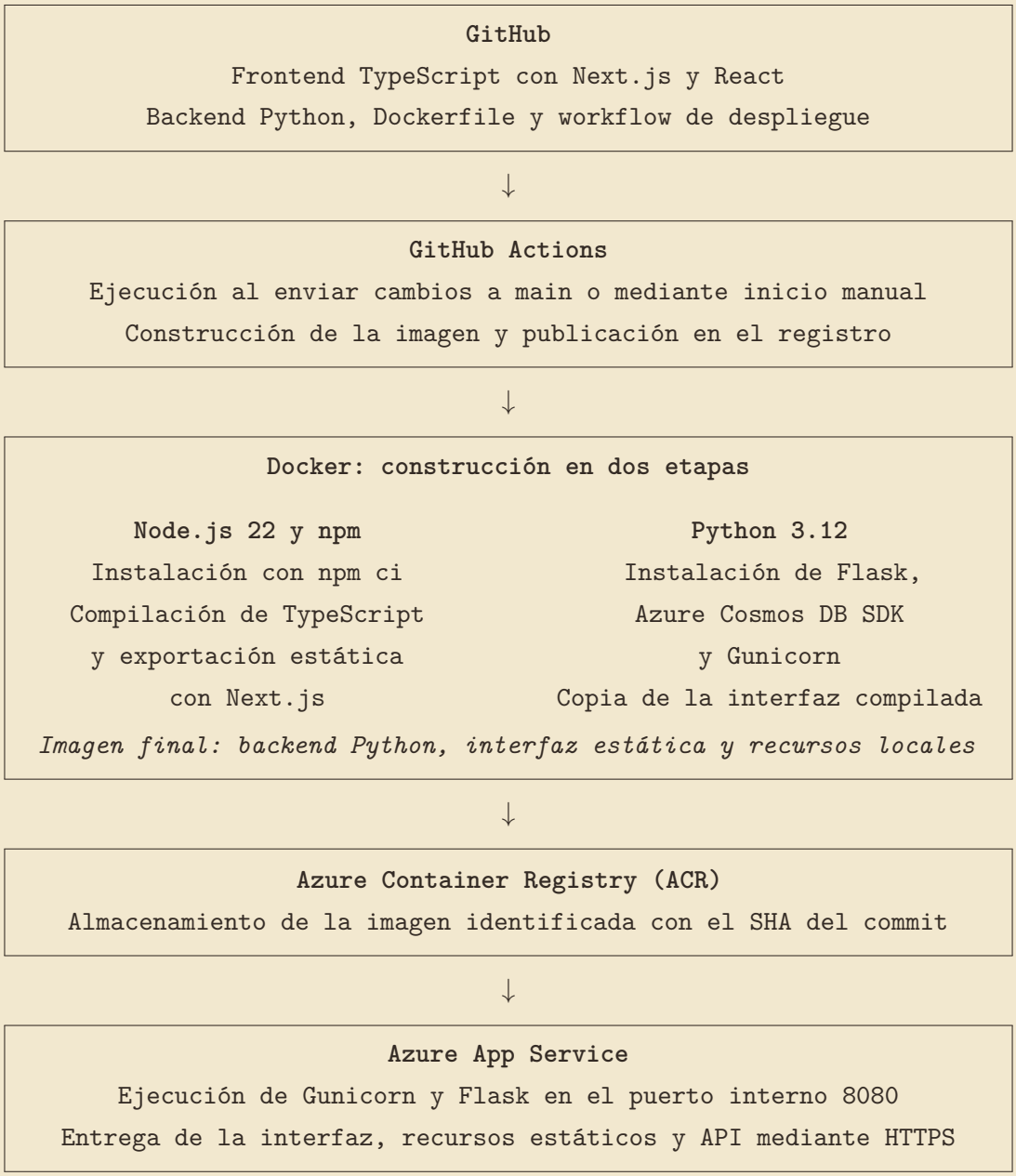
La aplicación web, identificada en la interfaz como **Typewriter**, ofrece una máquina de escribir para redactar y archivar apuntes de Ingeniería Civil, especialmente ideas de modelamiento matemático para una tesis. El proyecto conserva el nombre Recados en su API y en distintos recursos de infraestructura. Permite escribir textos de hasta 2000 caracteres, consultar los textos guardados en una cuenta de GitHub, abrirlos en el editor, guardar nuevas copias y eliminarlos de la colección personal (artemioadaysolvers, s. f.).

El usuario puede redactar y descargar el contenido del editor como un archivo .txt sin iniciar sesión. Para guardar en la nube o consultar su colección debe autenticarse con GitHub. Si inicia el acceso mientras está escribiendo, la interfaz intenta conservar el borrador en sessionStorage y recuperarlo al regresar a la misma pestaña. Después del acceso, el usuario debe revisar la cuenta seleccionada y pulsar nuevamente el botón de guardar; el borrador no se guarda automáticamente (artemioadaysolvers, s. f.).

La interfaz utiliza TypeScript, React y Next.js. Incluye un contador de palabras y caracteres, avisos de cambios sin guardar, el atajo Ctrl + S o Cmd + S, sonidos de teclado y botones, y un control para activarlos o desactivarlos. Su presentación utiliza una tipografía monoespaciada y ecuaciones decorativas servidas como archivos SVG locales. Estas ecuaciones son elementos visuales; la aplicación no implementa un motor de cálculo matemático (artemioadaysolvers, s. f.).

El backend está escrito en Python con Flask y utiliza Azure Cosmos DB para almacenar los textos de cada cuenta. La aplicación se empaqueta en una imagen Docker, que GitHub Actions construye y publica en Azure Container Registry. Azure App Service ejecuta el contenedor y expone la aplicación mediante HTTPS. Las credenciales privadas se suministran al backend mediante variables de entorno. El navegador recibe la interfaz compilada y los recursos públicos, mientras que las operaciones con GitHub y Cosmos DB que requieren secretos se realizan en el servidor (artemioadaysolvers, s. f.; Microsoft, s. f.-i).

Publicación de la aplicación Web



(artemioadaysolvers, s. f.; Docker, s. f.-c; Vercel, s. f.; GitHub, s. f.-c; Microsoft, s. f.-d)

Etapa 1. GitHub: Almacenamiento del código fuente. GitHub almacena los archivos del proyecto, incluyendo el frontend desarrollado con TypeScript, React.js y Next.js, el backend desarrollado en Python y los archivos de configuración necesarios para construir y publicar la aplicación web.

Etapa 2. GitHub Actions: Automatización del despliegue. GitHub Actions detecta las modificaciones incorporadas a la rama main y ejecuta automáticamente las instrucciones necesarias para construir y publicar una nueva versión de la aplicación en Microsoft Azure.

Etapa 3. Docker: Construcción de la aplicación. Docker construye la aplicación en dos etapas. En la primera, Node.js permite ejecutar Next.js para compilar el frontend y generar archivos HTML, CSS y JavaScript. En la segunda, se prepara el entorno Python y se incorporan los archivos del frontend compilado. El resultado es una imagen Docker que contiene los componentes necesarios para ejecutar la aplicación web.

Etapa 4. Azure Container Registry (ACR): Almacenamiento de la imagen Docker. Azure Container Registry almacena la imagen Docker generada durante la construcción, permitiendo que los servicios de Microsoft Azure puedan obtenerla para ejecutar la aplicación.

Etapa 5. Azure App Service (AAS): Ejecución y publicación de la aplicación. Azure App Service obtiene la imagen Docker almacenada en ACR y ejecuta el contenedor en la infraestructura de Microsoft Azure. Dentro del contenedor, el servidor Python distribuye los archivos del frontend, atiende las solicitudes de los usuarios y se comunica con Azure Cosmos DB para almacenar y recuperar sus textos. De esta manera, la aplicación queda disponible públicamente mediante una dirección web protegida con HTTPS.

Funcionamiento de la web

Navegador: Typewriter
Interfaz HTML, CSS y JavaScript (React.js)
Escritura, sonidos, contadores y descarga de textos

↓

Sin iniciar sesión: escribir y descargar localmente
Para guardar en la nube: iniciar sesión con GitHub

Backend Python: Flask
Recibe la solicitud de inicio de sesión
Prepara y protege el proceso de autenticación

↓

GitHub OAuth
Identifica al usuario mediante su cuenta de GitHub
Autoriza el acceso y devuelve un código al backend

↓

Backend Python: Verificación de identidad
Valida la respuesta de GitHub
Identifica al usuario y establece una sesión segura

↓

React recupera el borrador y carga la colección personal
El usuario confirma el guardado del texto

Backend Python: API Flask
Recibe solicitudes para consultar, guardar o eliminar textos
Verifica la sesión y los permisos del usuario

↓

Azure Cosmos DB
Almacena los textos asociados a cada usuario
Permite consultar, crear y eliminar registros

↓

Respuesta al navegador
Flask devuelve los resultados mediante HTTPS
React actualiza la colección sin recargar toda la página

(artemioadaysolvers, s. f.; Pallets, s. f.-c; Vercel, s. f.; GitHub, s. f.-c; Microsoft, s. f.-d)

Cuando un usuario accede a la aplicación web, el servidor desarrollado en Python mediante Flask proporciona los archivos HTML, CSS y JavaScript generados previamente por Next.js. El navegador interpreta estos archivos y ejecuta React.js para presentar la interfaz interactiva de la máquina de escribir. Al cargar la página, React consulta al backend mediante la ruta /api/session para comprobar si existe una sesión de usuario activa. Este procedimiento permite determinar si el usuario puede acceder a su colección personal de textos (artemioadaysolvers, s. f.; Pallets, s. f.-c; Vercel, s. f.).

La aplicación permite escribir, visualizar contadores de caracteres y palabras, reproducir sonidos y descargar textos en formato .txt sin necesidad de iniciar sesión. Estas operaciones se realizan directamente en el navegador mediante JavaScript y no requieren acceso a Azure Cosmos DB. Sin embargo, para almacenar los textos de manera permanente y recuperarlos posteriormente desde otros dispositivos, el usuario debe autenticarse mediante su cuenta de GitHub (artemioadaysolvers, s. f.).

La autenticación se realiza mediante el protocolo OAuth de GitHub. Cuando el usuario selecciona la opción de iniciar sesión o intenta guardar un texto sin estar autenticado, React conserva temporalmente el borrador en sessionStorage y redirige el navegador a la ruta /auth/login del backend. Flask prepara la solicitud mediante los mecanismos de seguridad state y PKCE con SHA-256, y redirige al usuario a GitHub para completar la identificación. Una vez autorizado el acceso, GitHub devuelve un código de autorización a la ruta /auth/callback (artemioadaysolvers, s. f.; GitHub, s. f.-c).

El backend comprueba la validez de la respuesta recibida, verifica que el proceso de autenticación no haya superado los diez minutos e intercambia el código de autorización por un token de acceso. Posteriormente, utiliza este token para consultar la identidad del usuario mediante la API de GitHub. A partir de su identificador numérico, genera un identificador interno mediante SHA-256 y establece una sesión de Flask asociada a esa cuenta. La sesión se mantiene mediante una cookie firmada, con una duración configurada de ocho horas, sin almacenar en ella el token de acceso de GitHub (artemioadaysolvers, s. f.; Pallets, s. f.-c; GitHub, s. f.-c).

Una vez completada la autenticación, el navegador regresa a la máquina de escribir. React comprueba nuevamente la sesión, recupera el borrador conserva-

do temporalmente y solicita al backend los textos asociados a la cuenta autenticada. El borrador recuperado no se guarda automáticamente: el usuario debe confirmar su almacenamiento mediante el botón correspondiente. La colección personal se obtiene mediante solicitudes HTTP a la API desarrollada con Flask (artemioadaysolvers, s. f.).

Cuando el usuario guarda, consulta o elimina un texto, React envía una solicitud al backend Python. Flask comprueba que exista una sesión válida y determina el identificador del propietario. Para las operaciones que modifican información, también verifica un token CSRF, destinado a proteger la aplicación frente a solicitudes no autorizadas realizadas desde otros sitios web. Posteriormente, el backend se comunica con Azure Cosmos DB mediante su biblioteca de Python para ejecutar la operación solicitada (artemioadaysolvers, s. f.; Pallets, s. f.-d; Microsoft, s. f.-d).

Azure Cosmos DB almacena cada texto junto con un identificador único, la fecha de creación y el identificador de su propietario. Esto permite separar las colecciones de los diferentes usuarios y consultar únicamente los textos asociados a la cuenta autenticada. Los registros se presentan del más reciente al más antiguo. Cuando se abre un texto guardado, su contenido se copia al editor; si posteriormente se modifica y guarda, se crea un nuevo registro en lugar de sobrescribir el original. Una vez completada cada operación, Flask devuelve el resultado al navegador y React actualiza la interfaz sin necesidad de recargar completamente la página (artemioadaysolvers, s. f.; Microsoft, s. f.-d).

Finalmente, la aplicación incorpora mecanismos adicionales de seguridad. Las credenciales utilizadas para GitHub OAuth y Azure Cosmos DB se mantienen en el backend mediante variables de entorno configuradas en Azure App Service, evitando incorporarlas al código JavaScript entregado al navegador. Las cookies de sesión utilizan las propiedades HttpOnly, SameSite=Lax y Secure bajo HTTPS, mientras que Flask establece cabeceras destinadas a restringir determinados comportamientos inseguros del navegador. De esta manera, la aplicación separa las operaciones de presentación, autenticación, procesamiento y almacenamiento de información (artemioadaysolvers, s. f.; Pallets, s. f.-a,s; Microsoft, 2023a).

Estructura del Repositorio de GitHub

```
Tutorial-Web-II-main/                    # Carpeta del repositorio
|-- .dockerignore                        # Exclusiones Docker
|-- .gitignore                           # Exclusiones Git
|-- Dockerfile                           # Construcción en dos etapas
|-- server.py                            # Flask, API y GitHub OAuth
|-- store.py                             # Operaciones Cosmos DB
|-- requirements.txt                     # Dependencias Python
|-- README.md                            # Presentación del proyecto
|
|-- .github/
|   `-- workflows/
|       `-- main_web-recados-2026abc.yml  # Construcción y despliegue
|
|-- frontend/                            # Next.js, React y TypeScript
|   |-- package.json                     # Dependencias y comandos
|   |-- package-lock.json                # Versiones resueltas
|   |-- next.config.mjs                  # Exportación estática
|   |-- next-env.d.ts                    # Declaraciones de Next.js
|   |-- tsconfig.json                    # Configuración TypeScript
|   `-- app/
|       |-- layout.tsx                   # HTML, idioma y metadatos
|       |-- page.tsx                     # Editor y colección personal
|       `-- globals.css                  # Estilos y diseño adaptable
|
|-- static/                              # Recursos servidos por Flask
|   |-- styles.css                       # Estilos de la página de error
|   |-- equations-heat.svg               # Ecuaciones decorativas
|   |-- equations-fields.svg             # Ecuaciones decorativas
|   `-- sounds/
|       |-- boton.wav                    # Sonido de botones
|       `-- teclado.wav                  # Sonido de escritura
|
|-- templates/
|   `-- error.html                       # Plantilla de errores
|
|-- tests/
|   |-- test_app.py                      # API, sesión y autenticación
|   |-- test_cosmos.py                   # Capa de almacenamiento
|   `-- test_export.py                   # Exportación estática y CSP
|
`-- docs/
    |-- AZURE.md                         # Configuración y publicación
    |-- VALIDACION.md                    # Validaciones documentadas
    `-- Tutorial.tex                     # Tutorial de configuración
```

(artemioadaysolvers, s. f.).

Principales Herramientas de JavaScript utilizadas

- **Node.js:** Entorno de ejecución de JavaScript que permite ejecutar programas desarrollados en este lenguaje fuera de los navegadores web. Se utiliza principalmente para desarrollar aplicaciones del lado del servidor y ejecutar herramientas de desarrollo, automatización y compilación (Node.js, s. f.).

- **React.js:** Biblioteca de JavaScript desarrollada para construir interfaces gráficas interactivas. Su principal característica es permitir que la presentación de una aplicación se actualice automáticamente cuando cambia la información que determina su contenido, evitando que el desarrollador tenga que modificar manualmente los elementos HTML correspondientes (React Team, s. f.).

- **Next.js:** Framework de desarrollo web basado en React.js que proporciona herramientas para organizar, compilar y optimizar aplicaciones web. Además, Next.js permite transformar el código desarrollado con React.js y TypeScript en archivos HTML, CSS y JavaScript que los navegadores pueden interpretar y ejecutar (Next.js, 2026).

En el proyecto, TypeScript y React.js se utilizan para desarrollar el código de la interfaz interactiva de la máquina de escribir. Posteriormente, Next.js permite organizar y compilar este código para generar archivos HTML, CSS y JavaScript que los navegadores pueden interpretar y ejecutar. Para realizar este proceso, Next.js utiliza Node.js como entorno de ejecución, ya que sus herramientas de desarrollo y compilación están diseñadas para funcionar con JavaScript fuera del navegador. Una vez finalizada la compilación, los archivos generados se incorporan a la imagen Docker de la aplicación, donde el servidor desarrollado en Python se encarga de distribuirlos a los navegadores de los usuarios y ejecutar las operaciones del backend, como la autenticación y la comunicación con la base de datos Azure Cosmos DB. De esta manera, Node.js interviene durante la construcción de la aplicación, mientras que Python se encarga de su funcionamiento en el servidor una vez publicada.

Tutorial

El siguiente tutorial describe el procedimiento de implementación y publicación de una aplicación web que simula una máquina de escribir de los años ochenta en Microsoft Azure, mediante el uso de contenedores Docker, almacenamiento en Azure Cosmos DB y autenticación con GitHub. La aplicación permite escribir textos mediante una interfaz inspirada en las máquinas de escribir tradicionales, reproducir sonidos de escritura, descargar documentos y almacenar una colección personal de textos asociados a cada usuario. Como etapa inicial, se establece la creación y verificación de una cuenta de Azure for Students, junto con la revisión de los créditos, las cuotas y la disponibilidad regional de los servicios requeridos.

La preparación del proyecto comprende la descarga del código fuente y su incorporación a un repositorio de GitHub mediante GitHub Desktop. Durante este proceso, se debe conservar la estructura original de los archivos y asegurar que el Dockerfile se encuentre en la raíz del repositorio. Asimismo, se define la rama main como origen para la publicación y se sincronizan los cambios con el repositorio remoto.

La configuración de la infraestructura contempla la creación de un grupo de recursos, un registro de contenedores en Azure Container Registry y una cuenta de Azure Cosmos DB para NoSQL. En esta última, se establecen la base de datos RecadosDB, el contenedor recados y la clave de partición /owner, utilizados para almacenar y organizar los textos guardados por los usuarios de la máquina de escribir. Posteriormente, se obtienen el URI y la clave de lectura y escritura necesarios para la conexión entre la aplicación y el servicio de almacenamiento.

El despliegue de la aplicación se realiza mediante Azure App Service, configurado con Linux y una imagen de contenedor temporal. En esta etapa, se obtiene la dirección pública del servicio, se habilita el acceso mediante HTTPS y se incorporan las direcciones IP salientes de App Service a las reglas de acceso de Cosmos DB. De manera complementaria, se registra una aplicación OAuth en GitHub para autenticar a los usuarios, utilizando la dirección pública de la aplicación y la ruta de retorno /auth/callback. Este mecanismo permite identificar a los usuarios y asociar los textos almacenados con sus respectivas

cuentas de GitHub.

Finalmente, se establecen las variables de entorno que contienen la configuración de ejecución, las credenciales de autenticación y los parámetros de conexión a Cosmos DB y el puerto de comunicación. La automatización del despliegue se configura mediante GitHub Actions desde el Centro de implementación de Azure, vinculando el repositorio, la rama de publicación y el registro de contenedores. De esta manera, las modificaciones realizadas en el código fuente pueden incorporarse automáticamente a la aplicación publicada. El procedimiento enfatiza la protección de las credenciales y su exclusión del código fuente.

Crear la cuenta de Estudiante

1. Entra en Azure for Students, selecciona **Comenzar gratis** y completa la verificación académica.

2. Abre portal.azure.com e inicia sesión.

3. En **Suscripciones**, comprueba que tienes seleccionada la suscripción correcta, por ejemplo **Azure for Students**.

Nombres que deben anotarse

Elemento	Ejemplo
Repositorio GitHub	Tutorial-Web-II
Grupo de recursos	rg-recados-web
Registro ACR	registrorecados2026abc
Repositorio de imágenes ACR	recados
Cuenta Cosmos DB	cosmosrecados2026abc
Base y contenedor de datos	RecadosDB y recados
Clave de partición	/owner
Web App	web-recados-2026abc
Registro de login Microsoft	recados-login
Rama de publicación	main

La URL pública se copia desde App Service. Puede tener sufijos adicionales. El repositorio de código de GitHub y el repositorio de imágenes de ACR son elementos distintos.

Descargar la Aplicación Web y subir el Proyecto a GitHub

Aplicación Web: artemioadaysolvers/Tutorial-Web-II.

Para descargar la aplicación web y subir sus archivos a un repositorio propio de GitHub, se utilizará **GitHub Desktop**, una aplicación que permite trabajar con repositorios Git mediante una interfaz gráfica, sin necesidad de ejecutar comandos desde una terminal (GitHub, s. f.-g).

1. Descargar e instalar **GitHub Desktop** desde https://desktop.github.com/. Una vez instalada la aplicación, iniciar sesión con una cuenta de GitHub.

2. Crear un repositorio para almacenar el proyecto. Existen dos alternativas:

   - **Crear un repositorio desde GitHub Desktop:** seleccionar **File** → **New Repository**, indicar el nombre del repositorio y su ubicación local. Posteriormente, seleccionar **Publish repository** para publicarlo en GitHub.

   - **Clonar un repositorio creado previamente desde la página web de GitHub:** si el repositorio ya fue creado manualmente en GitHub, seleccionar **File** → **Clone Repository**, elegir el repositorio vacío y presionar **Clone**. Esto creará una copia local del repositorio en el computador.

3. En GitHub Desktop, seleccionar el repositorio de destino y abrir **Repository** → **Show in Explorer**. Esto abrirá en el Explorador de archivos de Windows la carpeta local asociada al repositorio.

4. Acceder al repositorio de la aplicación web: artemioadaysolvers/Tutorial-Web-I. Seleccionar **Code** → **Local** → **Download ZIP** para descargar el proyecto completo en un archivo comprimido.

5. Descomprimir el archivo ZIP descargado. A continuación, copiar todos los

archivos y carpetas contenidos en el proyecto y pegarlos directamente en la carpeta local del repositorio abierta anteriormente mediante **Show in Explorer**. Se debe conservar la estructura original del proyecto y evitar introducir una carpeta adicional que contenga todos sus archivos.

6. Regresar a **GitHub Desktop**. En la pestaña **Changes** aparecerán automáticamente los archivos incorporados al repositorio. En el campo **Summary**, escribir una descripción de los cambios, por ejemplo, *Carga inicial de la aplicación web*. Posteriormente, presionar **Commit to main** para registrar los cambios en el repositorio local.

7. Finalmente, seleccionar **Push origin** para enviar los cambios registrados al repositorio remoto alojado en GitHub. Una vez finalizada la operación, abrir el repositorio desde la página web de GitHub y comprobar que todos los archivos y carpetas se encuentren correctamente publicados (GitHub, s. f.-g).

Es importante distinguir entre **Commit to main**, que registra los cambios en el repositorio local del computador, y **Push origin**, que envía dichos cambios al repositorio remoto de GitHub. Por el contrario, **Pull origin** permite incorporar al repositorio local los cambios disponibles en el repositorio remoto (GitHub, s. f.-g).

Crear el repositorio

1. Entrar en GitHub y seleccionar **New repository**.

2. Elegir propietario y nombre, por ejemplo Tutorial-Web-II.

3. Elegir **Public** o **Private**. En ambos casos, Azure debe recibir autorización para acceder al repositorio mediante la conexión con GitHub.

4. No agregar README, licencia ni .gitignore desde el asistente: el proyecto ya contiene los archivos necesarios.

5. Pulsar **Create repository** y copiar su URL real.

Si se reutiliza el repositorio de la calculadora, clonar ese repositorio, sustituir los archivos y retirar su antiguo despliegue. No borrar los recursos de Azure por cambiar el código.

Subir los archivos con GitHub Desktop

1. Instalar GitHub Desktop e iniciar sesión.

2. Abrir **File** → **Clone repository**, elegir el repositorio o pegar su URL, seleccionar una carpeta local y pulsar **Clone**.

3. Abrir **Repository** → **Show in Explorer**.

4. Copiar el contenido del proyecto descomprimido dentro de esa carpeta. Conservar la carpeta .git administrada por GitHub Desktop.

5. Revisar **Changes**: deben aparecer el Dockerfile, el backend y las carpetas de la aplicación.

6. Comprobar que la rama se llame main; si no, renombrarla desde las opciones de rama antes de conectar Azure.

7. Escribir un resumen y pulsar **Commit to main**.

8. Pulsar **Push origin** o **Publish branch**, según corresponda.

9. Abrir GitHub en el navegador y comprobar que el Dockerfile está en la primera lista de archivos.

Crear el grupo de recursos y Azure Container Registry

Crear el grupo de recursos

1. En Azure Portal, abrir **Grupos de recursos** y pulsar **Crear**.

2. Seleccionar la suscripción y escribir rg-recados-web en resource group

3. Elegir Chile Central si está disponible para la suscripción; de lo contrario, utilizar una región admitida y confirmar.

Crear el registro de imágenes

1. Buscar **Container registries / Registros de contenedores** y pulsar **Crear**.

2. Seleccionar resource group rg-recados-web, grupo y región; introducir el nombre único registrorecados2026abc.

3. En Ámbito de etiqueta de nombre de dominio seleccionar Tenant Reuse (Reutilización del inquilino).

4. Dejar uso de zonas de disponibilidad deshabilitado.

5. Elegir un nivel adecuado, por ejemplo **Basic**, revisar el precio y crear el recurso.

6. Seleccionar Plan de Precios Básico

7. En Modo de permisos de asignación de roles seleccionar Permisos del Registro RBAC

8. En Información general, copia el Servidor de inicio de sesión, que termina en .azurecr.io. Guárdalo para después.

Crear la cuenta de Azure Cosmos DB

1. Buscar **Azure Cosmos DB**, pulsar **Crear** y seleccionar **Azure Cosmos DB for NoSQL**.

2. En Tipo de carga de trabajo seleccionar Aprendizaje

3. Elegir resource group rg-recados-web, asignar el nombre único cosmosrecados2026abc y Ubicación Chile Central.

4. Seleccionar la capacidad conforme al presupuesto. Serverless y el nivel gratuito de rendimiento aprovisionado son modalidades diferentes; revisar la elegibilidad y el costo mostrado.

5. Asignar Modo de Capacidad con Rendimiento Aprovisionado, Aplicar descuento del nivel Gratis y Limitar la cantidad total de rendimiento que se puede aprovisionar en esta cuenta.

6. Seleccionar la pestaña Directiva de copia de seguridad y luego en Directiva de copia de seguridad asignar Continuo (7 días) (Este modo está disponible de forma gratuita.).

7. Revisar y crear el recurso.

Crear la base de datos y el contenedor en la cuenta de Azure Cosmos DB cosmosrecados2026abc

1. Abrir **Data Explorer / Explorador de datos** y seleccionar **New Container**.

2. Crear la base RecadosDB y el contenedor recados.

3. Escribir exactamente /owner como clave de partición.

4. Seleccionar el rendimiento Manual y Container Required RU/s a 400.

Obtener el URI y la clave de acceso de Azure Cosmos DB

1. Abrir **Keys / Claves** en el menú de la cuenta de Cosmos DB

2. Copiar el URI y una clave de lectura y escritura para usarlos después en la configuración web en App Service.

Crear App Service con Docker

Crear la Web App

1. Buscar **App Services** → **Crear** → **Aplicación web**.

2. Seleccionar suscripción , asignar el nombre único a la web web-recados -2026abc, asignar Contenedor en Publicar, Linux como sistema operativo, Región Chile Central.

3. Preocurar seleccionar plan de precios F1 Gratis.
4. En la pestaña **Contenedor**, desactivar la opción **Compatibilidad con Sidecar**. En **Origen de imagen**, seleccionar **Otros registros de contenedor** y elegir acceso **Público**. En **Servidor del registro**, escribir mcr.microsoft.com/k8se y, en **Imagen y etiqueta**, introducir quickstart:latest. Dejar vacío el campo **Comando de inicio**.

   Deja vacío el comando de inicio
5. Revisar la configuración y crear la aplicación.

Obtener la URL pública de la aplicación y configurar el acceso HTTPS

1. En Azure Portal, abrir **App Services** y seleccionar la aplicación web creada web-recados-2026abc. En el menú izquierdo, entrar en **Overview / Información general** y localizar el campo **Default domain / Dominio predeterminado**, dentro de **Aspectos básicos**. Copiar el dominio completo, incluidos sus sufijos, y agregar https:// al inicio para obtener la URL pública de la aplicación. Guardar esta URL para configurar posteriormente el inicio de sesión con Microsoft.

2. Abrir desde el menú **Configuración** y entrar en **Configuración general** o **Configuració (versión preliminar)**, según la interfaz disponible. Localizar la opción **HTTPS Only / Solo HTTPS**, establecerla en **Activado** y pulsar **Guardar** o **Aplicar**, si corresponde. Si ya está activada, conservar ese ajuste.

3. Por ahora, conservar la configuración de puerto de la imagen temporal. El puerto 8080 se establecerá al configurar las variables de la aplicación real, antes de su primer despliegue.

Permitir la conexión de App Service con Cosmos

1. En Azure Portal, abrir **App Services** y seleccionar la aplicación web-recados-2026abc. En el menú izquierdo, entrar en **Networking / Redes**. Dentro de **Configuración del tráfico saliente**, localizar y copiar la lista completa de **Direcciones IPv4 salientes**.

2. Abrir **Azure Cosmos DB**, seleccionar la cuenta cosmosrecados2026abc y entrar en **Networking / Redes** desde su menú izquierdo.

3. En **Acceso de red pública**, seleccionar **Redes seleccionadas**. En el apartado **Direcciones IP (IPv4 única o intervalo CIDR)**, agregar cada dirección IPv4 saliente de App Service como una entrada independiente. Conservar las reglas existentes que sigan siendo necesarias.

4. Pulsar **Guardar** y esperar a que se apliquen las reglas del firewall.

Las direcciones IP autorizadas corresponden a las conexiones salientes de App Service, no a los visitantes de la web. Los usuarios acceden a la aplicación mediante su navegador y el backend consulta Cosmos DB utilizando la clave de acceso configurada en las variables de entorno de App Service.

Como alternativa para una prueba temporal con datos ficticios, se puede seleccionar **Todas las redes** y guardar, sin agregar direcciones IP al firewall. Esta opción sigue exigiendo credenciales válidas para acceder a los datos, pero permite intentar conexiones desde cualquier dirección IP. Por ello, ofrece menor protección que **Redes seleccionadas**, que es la configuración utilizada en este tutorial.

Configurar el login con cuentas GitHub

La OAuth App identifica a los visitantes. Es independiente del repositorio de código y de las credenciales que el workflow utiliza para desplegar en Azure. Crear este registro con la cuenta personal de GitHub que administrará el proyecto; no se necesita registrar el login en Microsoft Entra ID.

1. Entrar en GitHub, pulsar la foto de perfil y abrir **Settings**. En el menú izquierdo, entrar en **Developer settings** → **OAuth Apps**. También se puede abrir https://github.com/settings/developers.

2. Pulsar **New OAuth App** o **Register a new application**, según la interfaz. Crear una **OAuth App**, no una GitHub App.

3. En **Application name**, escribir Recados. En **Application description**, se puede escribir "Aplicación para guardar recados personales".

4. En **Homepage URL**, pegar la URL pública de la aplicación web alojada en Azure. Para obtenerla, abrir **Azure Portal** → **App Services**, seleccionar la Web App y entrar en **Información general**. Copiar el valor completo de **Dominio predeterminado** y agregar https:// al inicio. Conservar todos los caracteres y sufijos que muestre Azure; no utilizar la dirección del repositorio de GitHub.

5. En la sección **Redirect URIs**, localizar el campo **Redirect URI** y pegar la URL pública completa de la Web App, agregando /auth/callback al final (Por ejemplo: [...]/auth/callback)). Esta es la dirección a la que GitHub devolverá al usuario después de autorizar el acceso. Dejar desmarcada la opción **Allow wildcard matching** y activada la opcion **Expire user access tokens**.

6. Pulsar **Register application**. En la página resultante, copiar el **Client ID** para configurarlo como GITHUB_CLIENT_ID en App Service.

7. En esa misma página, pulsar **Generate a new client secret**, completar la ve-

rificación de GitHub si se solicita y copiar el secreto generado para configurarlo como GITHUB_CLIENT_SECRET. Guardarlo en un lugar privado. Si ya no se puede consultar su valor, generar uno nuevo y actualizar App Service.

Configurar las variables de ejecución

Abrir App Service → **Configuración** → **Variables de entorno** → **Configuración de aplicación**. Agregar y guardar:

Nombre	Valor
SECRET_KEY	Cadena aleatoria estable de al menos 32 caracteres.
APP_URL	En Azure Portal, abrir **App Services**, seleccionar web-recados-2026abc y entrar en **Información general**. Copiar el **Dominio predeterminado** completo y agregar https:// al inicio. Pegar la URL resultante como valor de esta variable, sin barra final y sin agregar /auth/callback.
GITHUB_CLIENT_ID	En GitHub, pulsar la foto de perfil y abrir **Settings** → **Developer settings** → **OAuth Apps**. Seleccionar la aplicación Recados, copiar el valor de **Client ID** y pegarlo como valor de esta variable en App Service.
GITHUB_CLIENT_SECRET	En la misma OAuth App de GitHub, localizar **Client secrets** y pulsar **Generate a new client secret**. Completar la verificación si se solicita, copiar el secreto generado y pegarlo como valor de esta variable en App Service. Si ya se generó y se guardó su valor, utilizar ese secreto. No confundirlo con el **Client ID** ni con un token de acceso personal.
COSMOS_ENDPOINT	En Azure Portal, abrir **Azure Cosmos DB**, seleccionar la cuenta cosmosrecados2026abc y entrar en **Claves / Keys**. Copiar el valor completo del campo **URI** y pegarlo como valor de esta variable en App Service.

Nombre	Valor
COSMOS_KEY	En Azure Portal, abrir **Azure Cosmos DB**, seleccionar la cuenta cosmosrecados2026abc y entrar en **Claves / Keys**. En la pestaña **Claves de lectura y escritura**, copiar el valor de **Clave principal / Primary Key** y pegarlo como valor de esta variable en App Service. No utilizar una clave de solo lectura ni la cadena de conexión.
COSMOS_DATABASE	RecadosDB.
COSMOS_CONTAINER	recados.
WEBSITES_PORT	8080, para el modelo clásico.

Puedes generar la clave SECRET_KEY desde PowerShell ejecutando el siguiente código:

```
[Convert]::ToHexString([Security.Cryptography.RandomNumberGenerator]::GetBytes(32)).ToLower()
```

Copiar la cadena aleatoria de 64 caracteres que aparece como resultado y pegarla como valor de SECRET_KEY en las variables de entorno de App Service. Mantener el mismo valor entre despliegues e instancias y no subirlo a GitHub.

Configurar la publicación automática desde GitHub

Si el repositorio ya contiene un workflow que construye y publica esta Web App correctamente, conservarlo y pasar a la sección de comprobación. No generar otro por cambiar el proveedor del login. Si todavía se utiliza la imagen temporal y no existe automatización, seguir los pasos siguientes.

Habilitar las credenciales de administrador de Azure Container Registry

Para utilizar el registro de contenedores con la autenticación basada en credenciales de este tutorial, habilitar su usuario administrador:

1. En Azure Portal, buscar **Registros de contenedores / Container registries** y seleccionar el registro registrorecados2026abc.

2. En el menú izquierdo del registro, abrir **Claves de acceso / Access keys**.

3. Activar la opción **Usuario administrador / Admin user** y pulsar **Guardar**, si aparece. Si ya está activada, conservar ese ajuste.

4. Comprobar que se muestren el nombre de usuario y las contraseñas del registro. Utilizar estos valores únicamente en los campos de autenticación de Azure que los soliciten; no agregarlos al código ni subirlos al repositorio.

Estas credenciales permiten publicar y descargar imágenes del registro. Son independientes del GITHUB_CLIENT_ID y del GITHUB_CLIENT_SECRET, que se utilizan para el inicio de sesión de los visitantes.

Conectar GitHub desde Azure

1. En Azure Portal, abrir **App Services** y seleccionar la aplicación web web-recados-2026abc. En el menú izquierdo, entrar en **Configuración** → **Configuración general**.

2. Dentro de **Configuración de plataforma**, activar la opción **Credenciales de publicación de autenticación básica de SCM**. No es necesario activar las credenciales de publicación de FTP.

3. Pulsar **Guardar / Aplicar** y esperar a que se confirme el cambio. Después, abrir **Implementación** → **Centro de implementación** para continuar con la configuración del despliegue.

4. En el menú izquierdo, entrar en **Implementación** → **Centro de implementación**. Localizar el menú desplegable **Source / Origen** y seleccionar **GitHub Actions / Acciones de GitHub**, para configurar la construcción de la imagen Docker y su publicación automática a partir del código del repositorio.

5. Iniciar sesión en GitHub

6. Elegir organización o usuario, repositorio y rama main.

7. Elegir Agregar flujo de trabajo en **opción de flujo de trabajo**

8. Seleccionar Autenticación básica en **tipo de autenticación**

9. En **Origen de imagen**, seleccionar **Azure Container Registry** y, en **Registro**, elegir registrorecados2026abc.

10. En **Imagen**, escribir recados, que será el nombre del repositorio de imágenes de la aplicación dentro de Azure Container Registry.

11. pulsar **Guardar**.

Referencias

Adobe. (2021). *Adobe Flash Player EOL general information*. https://www.adobe.com/products/flashplayer/end-of-life.html

Apple. (s. f.-a). *AppKit*. Apple Developer Documentation. https://developer.apple.com/documentation/appkit

Apple. (s. f.-b). *SwiftUI*. https://developer.apple.com/swiftui/

Apple. (s. f.-c). *SwiftUI*. Apple Developer Documentation. https://developer.apple.com/documentation/swiftui

artemioadaysolvers. (s. f.). *Typewriter / Recados*. [Código fuente de la versión proporcionada por el autor en Tutorial-Web-II-main (4).zip: server.py, store.py, Dockerfile, workflow y carpeta frontend].

Ashley, K. (2015). *Microsoft Azure: Occasionally connected data in cross-platform mobile apps*. MSDN Magazine. https://learn.microsoft.com/en-us/archive/msdn-magazine/2015/january/microsoft-azure-occasionally-connected-data-in-cross-platform-mobile-apps

Autodesk. (s. f.). *Fusion API user's manual*. https://help.autodesk.com/cloudhelp/ENU/Fusion-360-API/files/UserManualIndex_UM.htm

Autodesk. (2026). *Creating an add-in*. https://help.autodesk.com/cloudhelp/2026/ENU/Inventor-API/files/CreatingAnAddIn_Overview.htm

Backus, J. (1978). *The history of FORTRAN I, II, and III*. *ACM SIGPLAN Notices, 13*(8), 165-180. https://doi.org/10.1145/960118.808380

Canonical. (s. f.-a). *Basic installation*. Ubuntu Server Documentation. https://ubuntu.com/server/docs/tutorial/basic-installation/

Canonical. (s. f.-b). *How to set up SSSD with Active Directory*. Ubuntu Server documentation. https://ubuntu.com/server/docs/how-to/sssd/with-active-directory/

Canonical. (s. f.-c). *OpenSSH server*. Ubuntu Server documentation. https://ubuntu.com/server/docs/how-to/security/openssh-server/

Canonical. (s. f.-d). *User management*. Ubuntu Server documentation. https://ubuntu.com/server/docs/how-to/security/user-management/

Canonical. (s. f.-e). *Web services*. Ubuntu Server Documentation. https://ubuntu.com/server/docs/how-to/web-services/

Centrum Wiskunde & Informatica. (2015). *25 years of Python at CWI*. https://www.cwi.nl/en/news/25-years-of-python-at-cwi/

CERN. (s. f.). *The birth of the Web*. https://home.cern/science/computing/the-birth-of-the-web

Chacon, S. (2009, 2 de enero). *Only the beginning*. The GitHub Blog. https://github.blog/news-insights/the-library/only-the-beginning/

Cleron, M. (2017). *Android announces support for Kotlin*. https://android-developers.googleblog.com/2017/05/android-announces-support-for-kotlin.html

Czerwonka, J., Nagappan, N., Schulte, W., & Murphy, B. (2013). *CODEMINE: Building a software development data analytics platform at Microsoft. IEEE Software, 30*(4), 64-71. https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/CodeMine.pdf

Debian Project. (2022). *Debian social contract (version 1.2)*. https://www.debian.org/social_contract.en.html

Do, T. N. (2026). *Zig computation benchmark*. Sharkbench. https://sharkbench.dev/computation/zig

Docker. (s. f.-a). *Building best practices*. https://docs.docker.com/build/building/best-practices/

Docker. (s. f.-b). *Docker Engine security*. Docker Docs. https://docs.docker.com/engine/security/

Docker. (s. f.-c). *Multi-stage builds*. https://docs.docker.com/build/building/multi-stage/

Docker. (s. f.-d). *What is an image?*. https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-an-image/

Docker. (s. f.-e). *What is Docker?* Docker Docs. https://docs.docker.com/get-started/docker-overview/

Ekins, B. (2018). *Understanding geometry and B-Rep in Inventor and*

Fusion 360. https://www.autodesk.com/autodesk-university/class/Understanding-Geometry-and-B-Rep-Inventor-and-Fusion-360-2018

Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022, junio). *HTTP semantics* (RFC 9110). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc9110

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley. https://martinfowler.com/books/eaa.html

Ginger Bill. (s. f.). *Overview: Odin programming language*. https://odin-lang.org/docs/overview/

GitHub. (s. f.-a). *About GitHub Desktop*. https://docs.github.com/en/desktop/overview/about-github-desktop

GitHub. (s. f.-b). *Acerca de los pull requests*. GitHub Docs. https://docs.github.com/es/pull-requests/get-started/about-pull-requests

GitHub. (s. f.-c). *Authorizing OAuth apps*. https://docs.github.com/en/apps/oauth-apps/building-oauth-apps/authorizing-oauth-apps

GitHub. (s. f.-d). *GitHub Docs*. https://docs.github.com/en

GitHub. (s. f.-e). *GitHub features*. https://github.com/features

GitHub. (s. f.-f). *GitHub-hosted runners reference*. https://docs.github.com/en/actions/reference/runners/github-hosted-runners

GitHub. (s. f.-g). *Hello world*. GitHub Docs. https://docs.github.com/en/get-started/using-github/hello-world

GitHub. (s. f.-h). *Managing your personal access tokens*. https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens

GitHub. (s. f.-i). *¿Qué es GitHub?*. GitHub Docs. https://docs.github.com/es/get-started/start-your-journey/what-is-github

GitHub. (s. f.-j). *Self-hosted runners*. https://docs.github.com/en/actions/concepts/runners/self-hosted-runners

GitHub. (s. f.-k). *Setting repository visibility*. https://docs.github.co

m/en/repositories/managing-your-repositorys-settings-and-features/managing-repository-settings/setting-repository-visibility

GitHub. (s. f.-l). *Workflows*. https://docs.github.com/en/actions/concepts/workflows-and-actions/workflows

GlebOlsen. (2026). *benchLANG: Programming language benchmark 2026* [Repositorio de código]. GitHub. https://github.com/GlebOlsen/benchLANG

Gomes, J. P., Monteiro, R., Moreira, A. S., Mendonça, V., Pinho, A., & Cunha, C. R. (2026). Teaching introductory programming in top universities: A global study of languages, paradigms, assessment, and AI. *Journal of Computer Languages, 86*, 101384. https://doi.org/10.1016/j.cola.2025.101384

Google. (s. f.-a). *Avoid non-composited animations*. https://developer.chrome.com/docs/lighthouse/performance/non-composited-animations

Google. (s. f.-b). *Develop UI*. Android Developers. https://developer.android.com/develop/ui

Google. (s. f.-c). *Jetpack Compose*. https://developer.android.com/compose

GTK Development Team. (s. f.). *Getting started with GTK*. GTK 4 Documentation. https://docs.gtk.org/gtk4/getting_started.html

Gunicorn Contributors. (s. f.). *Settings: Gunicorn*. https://gunicorn.org/reference/settings/

Hertzog, R., & Mas, R. (s. f.). *The Debian administrator's handbook*. https://www.debian.org/doc/manuals/debian-handbook/

Horn, R., Lahnaoui, A., Reinoso, E., Peng, S., Isakov, V., Islam, T., & Malavolta, I. (2023). Native vs web apps: Comparing the energy consumption and performance of Android apps and their web counterparts. En *2023 IEEE/ACM 10th International Conference on Mobile Software Engineering and Systems (MOBILESoft)* (pp. 44-54). IEEE. https://doi.org/10.1109/MOBILSoft59058.2023.00013

International Digital Economy Academy. (s. f.). *MoonBit*. https://www.moonbitlang.com/

Internet Security Research Group. (s. f.). *How it works*. Let's Encrypt. https://letsencrypt.org/how-it-works/

Jemerov, D. (2011). *Why JetBrains needs Kotlin*. https://blog.jetbrains.com/kotlin/2011/08/why-jetbrains-needs-kotlin/

Jullien, N., Clément-Fontaine, M., & Dalle, J.-M. (Coords.). (2002). *Nouveaux modèles économiques, nouvelle économie du logiciel*. [Informe final de investigación]. Projet RNTL. https://www.marsouin.org/IMG/pdf/fichier_rapport.pdf

Laforge, A. (2016). *Flash and Chrome*. https://blog.google/products-and-platforms/products/chrome/flash-and-chrome/

Lerner, J., & Tirole, J. (2002). *Some simple economics of open source*. *The Journal of Industrial Economics, 50*(2), 197-234. https://doi.org/10.1111/1467-6451.00174

Linux Kernel Development Community. (s. f.). *KVM*. The Linux Kernel documentation. https://docs.kernel.org/virt/kvm/index.html

MariuzM. (s. f.). *Bench: Head-to-head programming language benchmarks* [Repositorio de código]. GitHub. https://github.com/MariuzM/bench

MDN Web Docs. (s. f.-a). *HTML: HyperText Markup Language*. https://developer.mozilla.org/en-US/docs/Web/HTML

MDN Web Docs. (s. f.-b). *JavaScript*. https://developer.mozilla.org/en-US/docs/Glossary/JavaScript

MDN Web Docs. (s. f.-c). *What is a progressive web app?* Mozilla. https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/What_is_a_progressive_web_app

Mernik, M., Heering, J., & Sloane, A. M. (2005). *When and how to develop domain-specific languages*. *ACM Computing Surveys, 37*(4), 316-344. https://doi.org/10.1145/1118890.1118892

Metri, G., Shi, W., & Brockmeyer, M. (2015). Energy-efficiency comparison of mobile platforms and applications: A quantitative approach. En *Proceedings of the 16th International Workshop on Mobile Computing Systems and Applications* (pp. 39-44). Association for Computing

Machinery. https://doi.org/10.1145/2699343.2699358

Meyerovich, L. A., & Rabkin, A. S. (2013). Empirical analysis of programming language adoption. *ACM SIGPLAN Notices, 48*(10), 1-18. https://doi.org/10.1145/2509136.2509515

Microsoft. (s. f.-a). *A tour of C#*. https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/overview

Microsoft. (s. f.-b). *Active Directory Domain Services overview*. Microsoft Learn. https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/get-started/virtual-dc/active-directory-domain-services-overview

Microsoft. (s. f.-c). *Azure Container Registry Entra permissions and role assignments overview*. https://learn.microsoft.com/en-us/azure/container-registry/container-registry-rbac-built-in-roles-overview

Microsoft. (s. f.-d). *Azure Cosmos DB SQL API client library for Python*. https://learn.microsoft.com/en-us/python/api/overview/azure/cosmos-readme

Microsoft. (s. f.-e). *Azure Functions Host*. GitHub. https://github.com/Azure/azure-functions-host

Microsoft. (s. f.-f). *Azure Hybrid Benefit for Windows Server*. Microsoft Learn. https://learn.microsoft.com/en-us/windows-server/get-started/azure-hybrid-benefit

Microsoft. (s. f.-g). *C# language documentation*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/csharp/

Microsoft. (s. f.-h). *Componentes y límites del sistema de información de Azure*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/security/fundamentals/infrastructure-components

Microsoft. (s. f.-i). *Configure CI/CD to custom containers*. https://learn.microsoft.com/en-us/azure/app-service/deploy-ci-cd-custom-container

Microsoft. (s. f.-j). *Desencadenador HTTP de Azure Functions*. https://learn.microsoft.com/es-es/azure/azure-functions/functions-bindings-h

ttp-webhook-trigger

Microsoft. (s. f.-k). *Getting started with WinUI*. https://learn.microsoft.com/en-us/windows/apps/get-started/

Microsoft. (s. f.-l). *Hyper-V virtualization in Windows Server and Windows*. Microsoft Learn. https://learn.microsoft.com/en-us/windows-server/virtualization/hyper-v/overview

Microsoft. (s. f.-m). *Install the Web Server WEB1*. Microsoft Learn. https://learn.microsoft.com/en-us/windows-server/networking/core-network-guide/cncg/server-certs/install-the-web-server-web1

Microsoft. (s. f.-n). *Introduction to Azure Container Registry*. https://learn.microsoft.com/en-us/azure/container-registry/container-registry-intro

Microsoft. (s. f.-o). *Introduction to .NET*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/core/introduction

Microsoft. (s. f.-p). *JSX: TypeScript handbook*. https://www.typescriptlang.org/docs/handbook/jsx.html

Microsoft. (s. f.-q). *Linux virtual machines pricing*. Microsoft Azure. https://azure.microsoft.com/en-us/pricing/details/virtual-machines/linux/

Microsoft. (s. f.-r). *Manage system node pools in Azure Kubernetes Service (AKS)*. Microsoft Learn. https://learn.microsoft.com/en-us/azure/aks/use-system-pools

Microsoft. (s. f.-s). *Performance best practices for WebView2 apps*. https://learn.microsoft.com/en-us/microsoft-edge/webview2/concepts/performance

Microsoft. (s. f.-t). *Publish revisions with GitHub Actions in Azure Container Apps*. https://learn.microsoft.com/en-us/azure/container-apps/github-actions

Microsoft. (s. f.-u). *¿Qué es Azure?*. Microsoft Azure. https://azure.microsoft.com/es-es/resources/cloud-computing-dictionary/what-is-azure/

Microsoft. (s. f.-v). *¿Qué es Azure Linux Container Host para Azure Kubernetes Service (AKS)?*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/azure-linux/azure-linux-aks-overview

Microsoft. (s. f.-w). *¿Qué es Windows Server?*. Microsoft Learn. https://learn.microsoft.com/es-es/windows-server/get-started/overview

Microsoft. (s. f.-x). *¿Qué son las regiones de Azure?*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/reliability/regions-overview

Microsoft. (s. f.-y). *The basics: TypeScript handbook*. https://www.typescriptlang.org/docs/handbook/2/basic-types.html

Microsoft. (s. f.-z). *The history of C#*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/csharp/whats-new/csharp-version-history

Microsoft. (s. f.-aa). *The TypeScript handbook*. TypeScript Documentation. https://www.typescriptlang.org/docs/handbook/intro.html

Microsoft. (s. f.-ab). *Virtual machines in Azure*. Microsoft Learn. https://learn.microsoft.com/en-us/azure/virtual-machines/overview

Microsoft. (s. f.-ac). *What is Azure Notification Hubs?*. https://learn.microsoft.com/en-us/azure/notification-hubs/notification-hubs-push-notification-overview

Microsoft. (s. f.-ad). *Windows authentication overview*. Microsoft Learn. https://learn.microsoft.com/en-us/windows-server/security/windows-authentication/windows-authentication-overview

Microsoft. (s. f.-ae). *Windows container version compatibility*. Microsoft Learn. https://learn.microsoft.com/en-us/virtualization/windowscontainers/deploy-containers/version-compatibility

Microsoft. (s. f.-af). *Windows developer glossary*. Microsoft Learn. https://learn.microsoft.com/en-us/windows/apps/get-started/windows-developer-glossary

Microsoft. (s. f.-ag). *Windows Server pricing and licensing*. https://www.microsoft.com/en-us/windows-server/pricing

Microsoft. (s. f.-ah). *Windows Server pricing and licensing*. Microsoft. https://www.microsoft.com/en/windows-server/pricing

Microsoft. (2000). *Microsoft introduces highly productive .NET programming language: C#*. https://news.microsoft.com/source/2000/06/26/microsoft-introduces-highly-productive-net-programming-language-c/

Microsoft. (2008, 27 de octubre). *Microsoft unveils Windows Azure at Professional Developers Conference*. Microsoft News. https://news.microsoft.com/source/2008/10/27/microsoft-unveils-windows-azure-at-professional-developers-conference/

Microsoft. (2010). *Windows Azure general availability*. https://blogs.microsoft.com/blog/2010/02/01/windows-azure-general-availability/

Microsoft. (2022). *Ten years of TypeScript*. https://devblogs.microsoft.com/typescript/ten-years-of-typescript/

Microsoft. (2023a, 14 de noviembre). *Estrategias de arquitectura para proteger los secretos de aplicación*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/well-architected/security/application-secrets

Microsoft. (2023b). *Overview of .NET Framework*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/framework/get-started/overview

Microsoft. (2024, 26 de junio). *¿Qué es Azure Static Web Apps?* Microsoft Learn. https://learn.microsoft.com/es-es/azure/static-web-apps/overview

Microsoft. (2025a, 19 de septiembre). *Estilo de arquitectura de n niveles*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/architecture/guide/architecture-styles/n-tier

Microsoft. (2025b, 28 de noviembre). *Implementación en Azure App Service mediante Acciones de GitHub*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/app-service/deploy-github-actions

Microsoft. (2025c, 3 de abril). *Procedimientos recomendados para el diseño de la API web RESTful*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/architecture/best-practices/api-design

Microsoft. (2025d, 31 de octubre). *What is the Server Core installation option in Windows Server?*. Microsoft Learn. https://learn.microsoft.com/en-us/windows-server/administration/server-core/what-is-server-core

Microsoft. (2026a, 27 de abril). *Azure Cosmos DB: Base de datos para la era de IA*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/cosmos-db/overview

Microsoft. (2026b, 24 de abril). *Configuración de la aplicación de App Service o Azure Functions para usar el inicio de sesión con GitHub*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/app-service/configure-authentication-provider-github

Microsoft. (2026c, 29 de septiembre). *Estilo de arquitectura Web-Queue-Worker*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/architecture/guide/architecture-styles/web-queue-worker

Microsoft. (2026d, 25 de septiembre). *Idiomas admitidos en Azure Functions*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/azure-functions/supported-languages

Microsoft. (2026e, 18 de agosto). *Información general de App Service*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/app-service/overview

Microsoft. (2026f, 18 de septiembre). *¿Qué es Azure Functions?* Microsoft Learn. https://learn.microsoft.com/es-es/azure/azure-functions/functions-overview

Milichiewicz, R., & Badurowicz, M. (2025). Comparative analysis of cross-platform application development tools in terms of operating system integration. *Journal of Computer Sciences Institute, 36*, 357-364. https://doi.org/10.35784/jcsi.7831

MoonBit. (s. f.). *WebAssembly integration*. MoonBit Documentation. https://docs.moonbitlang.com/en/latest/toolchain/wasm/

MoonBit. (2024). *MoonBit adds JS backend, up to 25x faster than native JS*. MoonBit Blog. https://www.moonbitlang.com/blog/js-support

Nakashima, J., Atassi, H., & Thorpe, D. (2010, abril). Desarrollo e im-

plementación de aplicaciones de Windows Azure en Visual Studio 2010. *MSDN Magazine*. https://learn.microsoft.com/es-es/archive/msdn-magazine/2010/april/microsoft-azure-developing-and-deploying-azure-apps-in-visual-studio-2010

Nim Contributors. (s. f.). *Nim programming language*. https://nim-lang.org/

Nim Development Team. (s. f.). *Nim compiler user guide*. https://nim-lang.org/docs/nimc.html

Nisan, N., & Schocken, S. (2005). *The elements of computing systems: Building a modern computer from first principles*. MIT Press. https://mitpress.mit.edu/9780262640688/the-elements-of-computing-systems/

Node.js. (s. f.). *Introduction to Node.js*. https://nodejs.org/learn/getting-started/introduction-to-nodejs

npm. (s. f.). *npm ci*. https://docs.npmjs.com/cli/v11/commands/npm-ci/

Open Source Initiative. (s. f.). *The open source definition*. https://opensource.org/osd

OpenJS Foundation. (s. f.-a). *About Node.js*. https://nodejs.org/en/about/

OpenJS Foundation. (s. f.-b). *Don't block the event loop (or the worker pool)*. Node.js. https://nodejs.org/en/learn/asynchronous-work/dont-block-the-event-loop

Oracle. (2010). *Oracle completes acquisition of Sun*. https://www.sec.gov/Archives/edgar/data/1341439/000119312510015241/dex991.htm

OWASP Foundation. (s. f.). *Secure product design cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Secure_Product_Design_Cheat_Sheet.html

Pallets. (s. f.-a). *Configuration handling: Flask*. https://flask.palletsprojects.com/en/stable/config/

Pallets. (s. f.-b). *HTTP exceptions: Werkzeug*. https://werkzeug.palletsprojects.com/en/stable/exceptions/

Pallets. (s. f.-c). *Quickstart: Flask*. https://flask.palletsprojects.com/

en/stable/quickstart/

Pallets. (s. f.-d). *Security considerations: Flask*. https://flask.palletsprojects.com/en/stable/web-security/

Pereira, R., Couto, M., Ribeiro, F., Rua, R., Cunha, J., Fernandes, J. P., & Saraiva, J. (2017). Energy efficiency across programming languages: How do energy, time, and memory relate? En *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering* (pp. 256-267). Association for Computing Machinery. https://doi.org/10.1145/3136014.3136031

Peters, T. (2004). *PEP 20: The Zen of Python*. Python Enhancement Proposals. https://peps.python.org/pep-0020/

Python Software Foundation. (s. f.-a). *Design and history FAQ (Python 3.12)*. https://docs.python.org/3.12/faq/design.html

Python Software Foundation. (s. f.-b). *The Python standard library (Python 3.12)*. https://docs.python.org/3.12/library/index.html

Python Software Foundation. (s. f.-c). *The Python tutorial*. Python Documentation. https://docs.python.org/3/tutorial/

Python Software Foundation. (s. f.-d). *What is Python? Executive summary*. https://www.python.org/doc/essays/blurb/

Ragan-Kelley, J., Barnes, C., Adams, A., Paris, S., Durand, F., & Amarasinghe, S. (2013). *Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines*. En *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation* (pp. 519-530). ACM. https://people.csail.mit.edu/jrk/halide-pldi13.pdf

React Contributors. (s. f.-a). *Quick start*. https://react.dev/learn

React Contributors. (s. f.-b). *React DOM APIs*. https://react.dev/reference/react-dom

React Team. (s. f.). *Adding interactivity*. React. https://react.dev/learn/adding-interactivity

Rescorla, E. (2000, mayo). *HTTP over TLS* (RFC 2818). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc2818

Rescorla, E. (2018, agosto). *The Transport Layer Security (TLS) protocol version 1.3* (RFC 8446). Internet Engineering Task Force. h t t p s : //www.rfc-editor.org/rfc/rfc8446

Shrestha, N., Botta, C., Barik, T., & Parnin, C. (2020). Here we go again: Why is it difficult for developers to learn another programming language? In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering* (pp. 691-701). Association for Computing Machinery. https://doi.org/10.1145/3377811.3380352

SixtyFPS GmbH. (s. f.). *Slint: Declarative GUI for Rust, C++, JavaScript and Python*. https://slint.dev/

Slint. (s. f.). *Web*. Slint Documentation. https://docs.slint.dev/latest/docs/slint/guide/platforms/web/

Supreme Court of the United States. (2021). *Google LLC v. Oracle America, Inc., 593 U.S. 1*. https://www.supremecourt.gov/opinions/20pdf/18-956_d18f.pdf

Szekeres, L., Payer, M., Wei, T., & Song, D. (2013). *SoK: Eternal war in memory*. En *2013 IEEE Symposium on Security and Privacy* (pp. 48-62). IEEE. https://doi.org/10.1109/SP.2013.13

Tanenbaum, A. S., & Wetherall, D. J. (2012). *Redes de computadoras*. (5.ª ed.). Pearson Educación.

The GNOME Project. (s. f.). *GNOME developer documentation*. https://developer.gnome.org/documentation/

The Linux Kernel Organization. (s. f.). *Linux kernel licensing rules*. The Linux Kernel Documentation. https://www.kernel.org/doc/html/latest/process/license-rules.html

The Qt Company. (s. f.-a). *All overviews*. Qt for Python Documentation. h t tps://doc.qt.io/qtforpython-6/overviews/qtdoc-overviews.html

The Qt Company. (s. f.-b). *Introduction to Qt*. https://doc.qt.io/qt-6/qt-intro.html

Torano, E. (2017). *Faster command line tools in Nim*. Nim Blog. https://nim-lang.org/blog/2017/05/25/faster-command-line-tools-in-nim.html

Turner, J. (2014, 2 de abril). *Announcing TypeScript 1.0*. Microsoft Developer Blogs. https://devblogs.microsoft.com/typescript/announcing-typescript-1-0/

Van Rossum, G. (s. f.). *Python's early days*. https://legacy.python.org/doc/essays/ppt/jpf001/tsld006.htm

Vercel. (s. f.). *How to create a static export of your Next.js application*. https://nextjs.org/docs/app/guides/static-exports

W3C. (s. f.). *A brief history of CSS until 2016*. https://www.w3.org/Style/CSS20/history

W3C. (2026). *File API*. https://www.w3.org/TR/2026/WD-FileAPI-20260912/

Wanstrath, C. (2008, 10 de abril). *We launched*. The GitHub Blog. https://github.blog/news-insights/we-launched/

WHATWG. (2026a). *Fetch: Living standard*. https://fetch.spec.whatwg.org/

WHATWG. (2026b, 19 de octubre). *HTML: Living standard*. https://html.spec.whatwg.org/multipage/

Wirfs-Brock, A., & Eich, B. (2020). *JavaScript: The first 20 years*. *Proceedings of the ACM on Programming Languages, 4*(HOPL), artículo 77, 1-189. https://wirfs-brock.com/allen/posts/866

World Wide Web Consortium. (2014). *HTML5: A vocabulary and associated APIs for HTML and XHTML*. W3C Recommendation. https://www.w3.org/TR/2014/REC-html5-20141028/

Zig Software Foundation. (s. f.). *Overview: Zig programming language*. https://ziglang.org/learn/overview/